

中国研究生操作系统开源创新大赛

项目说明书

作品名称:	面向 RISC-V Vector 扩展的 大语言模型推理并发调度优化
参赛队伍:	芯火燎原
完成日期:	2026 年 7 月

目录

第 1 章 绪论	1
1.1 赛题背景	1
1.2 现状及问题	1
1.2.1 现状	1
1.2.2 存在问题	2
1.3 项目主要贡献	3
1.4 实验平台与技术选型	3
第 2 章 技术理论	5
2.1 设计思想	5
2.1.1 问题分析	5
2.1.2 量化结果	8
2.1.3 优化策略	11
2.2 技术路线	12
2.3 代码原创声明	14
2.3.1 V 状态追踪脚本	14
2.3.2 性能评测脚本	14
2.3.3 自动化集成测试框架	14
2.3.4 llama.cpp 推理框架	14
第 3 章 项目介绍	15
3.1 总体设计	15
3.1.1 系统架构	15
3.1.2 接口关系	16
3.2 Trace 脚本模块	17
3.2.1 追踪原理	17
3.2.2 数据格式	18
3.3 量化分析模块	18
3.3.1 工作流程	18
3.3.2 量化指标	19
3.4 优化模块	20
3.4.1 粗粒度优化方案	20
3.4.2 sched_ext 自定义调度	23

3.5	性能评测模块	30
3.5.1	模块执行流程	30
3.5.2	采集指标与输出	32
第 4 章	实验测试	34
4.1	测试环境	34
4.1.1	硬件环境	34
4.1.2	软件环境	36
4.2	测试说明	37
4.3	测试步骤	38
4.3.1	模块测试	38
4.3.2	集成测试	39
4.4	测试结果	40
4.4.1	模块测试	40
4.4.2	集成测试	43
第 5 章	总结与展望	46
5.1	总结	46
5.2	未来工作	49
参考文献		50
附录 A	RISC-V 与 x86 架构向量寄存器切换的对比	52
A.1	扩展状态管理机制	52
A.2	观测环境与统计口径	53
A.3	观测解释与适用边界	53
附录 B	新内核与根文件系统适配过程	55
B.1	根文件系统适配	55
B.1.1	适配边界与输入材料	55
B.1.2	分区与根文件系统部署	55
B.1.3	验证方法	56
B.2	新内核适配与 sched_ext 启用	57
B.2.1	配置目标与编译输入	57
B.2.2	内核配置与编译	57
B.2.3	产物部署与启动项切换	58
B.2.4	验证方法	59
B.2.5	复现边界与维护要求	59

第 1 章 绪论

1.1 赛题背景

大语言模型（Large Language Model, LLM）在自然语言理解、代码生成、智能问答和文本摘要等任务中展现出较强的通用能力。随着低比特量化、算子优化和轻量化推理框架的持续发展，LLM 推理逐步由数据中心向边缘设备拓展。相较于云端部署，边缘侧推理能够缩短数据传输链路，并在一定程度上降低网络依赖，更好地适应智能家居、工业控制和机器人等场景对实时性、部署成本及数据隐私的要求。

RISC-V 采用开放、模块化的指令集体系，能够通过标准扩展面向不同计算场景构建处理器。其中，RISC-V Vector Extension（RVV）以向量长度无关的编程模型提供数据级并行能力，为边缘侧张量计算提供了体系结构基础^[1]。在软件生态方面，以 llama.cpp 为代表的轻量化推理框架已支持 RISC-V 向量扩展，并可利用 RVV 执行量化计算和矩阵运算等核心操作^[2-3]。

在算子向量化和模型量化已具备基本可用性的基础上，RISC-V 平台上的 LLM 推理优化还需要进一步关注系统层面的资源管理问题。尤其在多请求并发场景下，推理线程之间以及推理线程与系统任务之间会形成更复杂的处理器资源竞争。向量线程被抢占或迁移时可能触发向量上下文的保存与恢复，其开销会随调度行为累积，并可能影响服务吞吐率和尾延迟。因此，研究向量状态、线程调度与推理性能之间的关系，对于完善 RISC-V 边缘推理的软件栈具有现实意义。

1.2 现状及问题

1.2.1 现状

RISC-V 特权架构在状态寄存器中设置 VS（Vector State）字段，用于描述当前向量上下文的状态。VS 可取 Off、Initial、Clean 和 Dirty 四种状态；当向量指令修改了向量寄存器或相关控制状态后，VS 将被置为 Dirty^[1]。Linux 内核据此管理线程的向量上下文：对于未启用或无须保存向量状态的线程，可避免不必要的寄存器组保存；当被换出线程的向量状态为 Dirty 时，调度切换路径需要保存其向量上下文，以保证该线程再次运行时能够恢复正确的执行状态^[4]。

上述机制能够避免对所有线程无条件保存向量寄存器，但其实际收益取决于任务的向量使用特征。LLM 推理线程在执行矩阵乘法、量化解码等操作时会持续调用 RVV 优化算

子，向量状态因而更容易在调度发生前处于 **Dirty** 状态。若此类线程频繁被抢占或在处理器核心之间迁移，向量上下文管理所产生的附加开销便可能在长时间运行和高并发条件下逐步累积。

LLM 自回归推理通常由 **Prefill** 和 **Decode** 两个阶段构成。**Prefill** 阶段对输入序列进行批量计算，具有较高的数据并行度；**Decode** 阶段则逐 **token** 迭代执行，并反复调用张量算子。尽管两个阶段的计算特征并不完全相同，其执行过程均可能长期占用向量计算资源。Linux 6.6 的默认公平调度器主要依据任务权重、虚拟运行时间和系统负载进行处理器分配^[5]，并不显式感知线程的 **VS** 状态、向量上下文管理成本或推理阶段特征。因此，通用调度决策与向量密集型推理负载之间可能存在优化空间。

1.2.2 存在问题

目前，RISC-V Linux 已具备向量状态管理和通用任务调度能力，但面向 LLM 并发推理的 **V** 状态观测、开销量化及调度优化仍缺少可直接复用的完整方法。结合 llama.cpp server 的执行模型，现有系统主要面临以下问题。

首先，Linux 通用调度器面向多种负载提供公平性与吞吐能力，其调度决策并不以任务当前的 **VS** 状态或向量上下文管理成本为直接依据。当多个可运行任务共享处理器核心时，处于 **Dirty V** 状态的推理线程可能被换出；若该现象持续发生，向量状态相关操作将形成额外的系统开销。由于内核中的实际保存函数难以直接挂载常规动态探针，还需要建立与调度路径相匹配的代理观测方法，才能对相关事件进行可靠归因。

其次，llama.cpp server 通过 slot、批次和工作线程协同处理并发请求^[3,6-7]。请求并发数增加时，同时处于活跃状态的推理任务随之增多，但计算线程规模主要由启动参数控制，并不与请求数保持简单的线性关系。当活跃请求超过现有计算资源的承载能力时，请求排队、线程竞争和调度等待会相互叠加，使单纯依靠应用层参数难以解释性能变化。

最后，实际服务不仅包含执行张量计算的工作线程，还包含网络请求处理、控制逻辑和结果返回等通用任务。这些线程与操作系统后台任务共同竞争 CPU 资源，可能干扰推理线程的连续执行。若仅采用进程级绑定而缺少更细粒度的资源隔离，推理线程仍可能受到同一核心上其他任务的抢占；若采用实时调度等强干预策略，又需要兼顾系统公平性、交互响应和异常回退。因此，调度方案需要在性能收益、系统可用性和部署复杂度之间进行权衡。

针对上述问题，本项目从观测、优化和验证三个层面开展研究。首先，构建 RISC-V 向量上下文切换的代理观测与量化方法，分析模型规模、请求并发度、调度行为与推理性能之间的关系；其次，结合 CPU affinity、cgroup cpuset、Linux 调度类和 sched_ext 可编程调

度机制，设计不同干预强度的资源放置与调度方案；最后，在支持 RVV 1.0 的真实硬件平台上部署 llama.cpp server，采用吞吐率、平均首 token 延迟和 P95 首 token 延迟等指标，对各方案的运行行为及适用边界进行实验评估。

1.3 项目主要贡献

本项目围绕 RISC-V V 状态感知的 LLM 推理调度优化，完成了观测工具、调度方案和自动化评测流程的设计与实现，主要工作和贡献如下。

在观测与方案评估方面，本项目基于 bpfttrace、kprobe 和 tracepoint 构建调度路径观测工具，在 sched_switch 事件处读取被换出线程的 VS 状态快照，统计满足 Dirty-V 保存条件的代理事件，并按照线程和 CPU 进行归类。该方法不将代理事件等同于硬件层面的真实保存次数，而是用于刻画向量密集型线程被换出的调度行为。在此基础上，项目对 taskset、cpuset cgroup、partition root、systemd-scope、SCHED_BATCH 和 SCHED_FIFO 六种 CPU 放置、资源隔离与调度配置进行对比，分析其对推理性能和 V 状态条件事件的影响。

在自定义调度方面，本项目基于 sched_ext 实现面向 llama.cpp 的可编程调度原型。该调度器识别目标进程中的推理工作线程，将其限制在 SpacemiT K1 的 CPU 0–3 AI 计算核心上运行，并采用 30 ms 推理时间片和原核心优先的选核策略，以减少非必要的调度轮转与跨核迁移。对于普通任务，调度器将其引导至 CPU 4–7 的通用核心域，并通过独立派发队列实现计算任务与系统任务的资源隔离。

在实验与工程化方面，本项目以 Milk-V Jupiter 为主要实验平台，选用 Llama 3.2 1B 和 3B 的 Q4_K_M 量化模型，在 2、4、8 并发条件下采集推理吞吐率、首 token 延迟和端到端延迟等指标。项目进一步将服务启动、策略部署、并发请求、性能采集、V 状态追踪、环境清理与 CSV 归档整合为自动化评测流水线，使不同调度配置能够在统一流程和实验口径下重复执行与比较。

1.4 实验平台与技术选型

本项目选用 Milk-V Jupiter 开发板作为主要实验平台，如图 1.1 所示。该开发板搭载 SpacemiT K1 RISC-V SoC，处理器包含 8 个 64 位 X60 CPU 核，实验环境中的默认主频为 1.6 GHz。其中，CPU 0–3 作为具备向量计算能力的 AI 计算核心，用于承载 llama.cpp 推理工作线程；CPU 4–7 作为通用核心，用于运行服务控制线程和系统任务。K1 支持 RISC-V Vector Extension 1.0 (RVV 1.0)，向量寄存器长度为 256 bit (VLEN=256)，能够执行 llama.cpp 量化推理中的向量化矩阵计算。

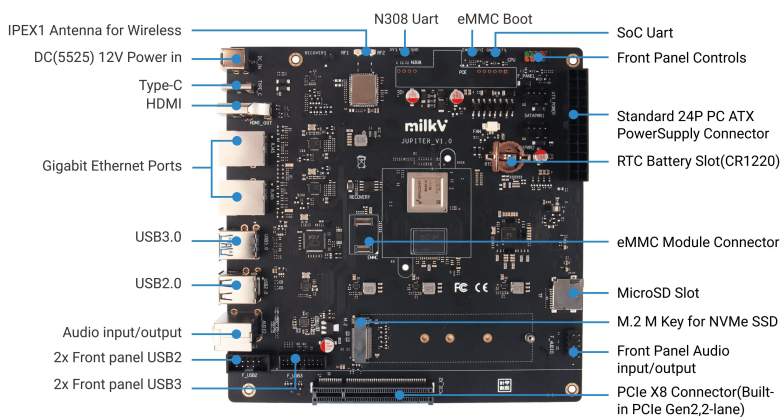


图 1.1 Milk-V Jupiter 开发板

实验所用 Jupiter 开发板配置 8 GB LPDDR4X 内存，并通过 M.2 M-Key 接口连接 NVMe 固态硬盘，用于存放 GGUF 量化模型、测试脚本和实验结果。开发板提供的千兆以太网、USB 3.0 和串口等接口分别承担请求传输、外设连接与系统调试功能，从而支持模型部署、远程控制、日志回传和重复实验。

软件环境以 Linux 6.6 内核为基础，部署 RISC-V 编译工具链、性能分析工具和远程实验组件。推理框架采用启用 RVV 支持的 llama.cpp server 模式，正式评测模型为 Llama 3.2 1B Instruct 和 3B Instruct，均使用 Q4_K_M 量化格式^[8-9]。该软硬件组合使推理负载、线程调度和内核观测能够在同一真实 RVV 1.0 平台上运行，为后续开展 V 状态条件事件分析与调度方案对比提供统一的实验基础。

第 2 章 技术理论

本章围绕 RISC-V 向量密集型大语言模型并发推理中的调度开销展开理论分析与方案设计。首先，从 Linux 调度机制、RISC-V V 状态管理以及 llama-server 请求执行流程三个层面阐明问题的形成机理；继而构建量化观测方法，分析调度行为、向量上下文条件事件与推理性能之间的关系；在此基础上提出资源放置、运行隔离和可编程调度等优化策略，并给出项目的总体技术路线，为后续系统实现与实验验证奠定理论基础。

2.1 设计思想

本节从 Linux 通用调度机制、RISC-V V 状态管理和 llama-server 请求执行链三个层面分析调度开销的形成机理，并在此基础上开展量化观测，提出相应的优化策略。

2.1.1 问题分析

问题分析依次考察 Linux 调度类的运行机制、向量上下文的保存条件以及推理服务内部的任务流转过程。

2.1.1.1 Linux 调度类

Linux 调度器通过多个调度类管理具有不同实时性和资源需求的任务。执行调度决策时，内核首先检查优先级较高的可运行调度类，再在选定的调度类内部确定具体任务。停止类主要供内核执行 CPU 停止和任务迁移等内部操作；空闲类承载每个 CPU 的空闲线程，仅在不存在其他可运行任务时执行。普通用户态程序主要涉及截止期类、实时类和公平调度类^[5]。

公平调度类是 Linux 中最常用的调度类，其传统实现机制通常称为 CFS。普通用户程序默认采用 SCHED_OTHER 策略，由公平调度器依据任务权重分配 CPU 时间；任务的 nice 值会影响其调度权重，内核则通过虚拟运行时间等信息避免单个任务长期占用处理器，其中 `vruntime` 是衡量任务公平性的关键依据^[5]。Linux 6.6 的公平调度路径已采用 EEVDF 机制选择任务，但仍延续 CFS 的基本目标，即在普通任务之间维持相对公平，并兼顾任务唤醒后的响应性与系统整体吞吐^[10]。llama-server 及其推理线程默认均运行在这一公平调度路径中。

公平调度类还提供 SCHED_BATCH 和 SCHED_IDLE 两种策略。SCHED_BATCH 面向吞吐优先、交互性要求较低的 CPU 密集型任务，可降低任务唤醒后抢占其他任务的倾向，

适用于离线批处理或后台计算；`SCHED_IDLE` 的调度优先级更低，主要适用于仅需在系统空闲时执行的后台任务^[11]。

截止期类对应 `SCHED_DEADLINE`，适用于具有明确运行预算、执行周期和完成期限的硬实时任务。该调度类能够提供较强的时间约束，但要求任务的执行时间与周期具有较好的可预测性。由于 LLM 推理请求的到达时间、上下文长度和生成长度均可能动态变化，因此通常不宜直接采用该策略。实时类包括 `SCHED_FIFO` 和 `SCHED_RR`。`SCHED_FIFO` 适用于需要持续获得 CPU 并尽量避免普通任务抢占的关键任务；同优先级任务不会因时间片耗尽而自动轮转，而是持续运行至主动让出、进入阻塞状态或被更高优先级的实时任务抢占。`SCHED_RR` 则适用于多个同优先级实时任务按照时间片轮转执行的场景^[11]。

除了这些默认调度策略之外，`SCHED_EXT` 是 Linux 提供的可编程调度扩展框架，用于在不直接修改内核通用调度器源码的前提下，通过 BPF 程序定义特定任务的调度行为。与 CFS、实时类等固定调度类相比，`sched_ext` 允许调度程序在任务唤醒、入队、选核、派发和时间片分配等关键节点参与决策^[12]。

2.1.1.2 Linux 对 V 状态的管理

Linux 内核通过 `sstatus` 寄存器中的 VS 字段管理各线程的向量寄存器状态，VS 共有四种取值。线程创建后，若尚未执行向量指令，VS 处于 Off 状态；当线程启用向量扩展后，内核为其建立向量上下文，并将 VS 置为 Initial，此时寄存器组处于体系结构规定的初始状态。

线程执行向量指令并修改寄存器内容后，VS 转为 Dirty，表明当前向量上下文已发生变化；若此时发生上下文切换，内核需要保存相应状态，并在保存后将 VS 置为 Clean。若线程从被换入到再次换出期间未修改向量状态，VS 将保持为 Clean，调度器在换出时无须重复执行保存操作。各状态的编码、含义及切换行为如表 2.1 所示。

表 2.1 VS 状态编码及切换时内核行为

状态	编码	含义	切换时内核行为
Off	00	未使用向量	无操作
Initial	01	已初始化，数据无意义	仅设 lazy restore 标志
Clean	10	数据有效但未修改	lazy restore，首次向量指令时硬件自动加载
Dirty	11	数据已被修改	全量保存 v0-v31 到内存（256-bit × 32 = 1024 字节）

2.1.1.3 llama-server 请求处理流程

llama-server 将一次文本生成请求依次转换为任务、逻辑会话槽和模型计算批次，最终通过普通 HTTP 响应或流式响应将采样结果返回客户端。需要说明的是，请求、slot 与推理线程之间并非一一对应：一次 HTTP 请求首先被转换为一个任务；任务占用一个 slot，以维护独立的上下文和 KV 状态；实际的矩阵计算则由共享的 GGML 线程池完成^[3,6]。

服务端接收 *completion* 请求后，首先解析 JSON 请求体，将 *messages* 转换为统一的 *prompt* 表示，并处理请求中可能携带的多模态文件。随后，服务端解析生成长度、采样参数、停止词和流式标志等配置，使用模型词表将输入转换为 token 序列，并为每个输入创建包含 token、采样配置、响应格式和唯一编号的独立任务^[6]。

任务进入服务端队列后，推理循环根据 slot 的空闲状态、任务指定的 slot 以及 *prompt* 缓存相似度分配逻辑 slot；暂时无法获得可用 slot 的任务则进入等待队列。获得 slot 后，服务端检查输入长度、上下文上限和可复用的 *prompt* 缓存，将尚未计算的 *prompt* token 加入本轮 *server batch*，并为每个 token 记录位置、*sequence id* 和 *logits* 输出标志。启用 *continuous batching* 后，同一 *batch* 可同时容纳多个请求的 *decode token* 与 *prompt token*^[3,6]。

进入计算阶段后，不同请求继续通过 *sequence id* 和 *position* 隔离各自的 KV 状态。构造完成的逻辑 *batch* 被传入 *llama_decode()*，并依据物理 *micro-batch* 的容量上限划分为一个或多个 *ubatch*，依次完成 KV memory 分配、计算图构建或复用、输入张量绑定、后端计算和 *logits* 提取。CPU 后端中的 GGML 根据 *ubatch* 的 token 数选用 *-t* 或 *-tb* 对应的线程配置；共享推理线程按照矩阵计算任务分工，并不与某个 slot 或特定的 *Prefill*、*Decode* 过程固定绑定^[3,7]。

Prompt 的最后一个 token 完成计算后，服务端根据对应的 *logits* 进行采样，将生成 token 写入该 slot 的结果，并在下一轮将其作为新的 *decode* 输入。上述循环持续至触发 EOS、停止词、最大生成长度、上下文容量或时间限制。流式请求逐步返回 *partial response*，非流式请求则等待完整结果；生成结束后，服务端返回生成文本、停止原因、token 数和推理计时信息，释放相应 slot，并唤醒等待队列中的后续任务^[6]。

在默认配置下，llama-server 的控制线程和推理线程均采用 *SCHED_OTHER* 策略，运行于 Linux 公平调度路径。推理线程在执行模型矩阵算子时会频繁使用 *RVV* 指令，其运行、抢占、迁移以及重新获得 CPU 的过程均由内核调度器决定^[5,7,10-11]。

2.1.1.4 结论

为避免在每次上下文切换时无条件保存向量寄存器，RISC-V Linux 通过 *sstatus.VS* 的 *Off*、*Initial*、*Clean* 和 *Dirty* 四种编码追踪向量上下文状态^[4]。仅当被换出线程的向

量状态为 Dirty 时，切换路径才需要保存相应的寄存器组和控制状态。对于向量指令使用频率较低的通用负载，该机制能够避免不必要的保存操作；而在 LLM 推理等向量密集型场景中，推理线程更容易在被换出时保持 Dirty 状态，按需保存操作因而可能被频繁触发。

LLM 推理的执行特征使默认调度机制同时面临时间维度和空间维度的优化问题。

在时间维度上，llama-server 处理文本生成请求时会在矩阵计算区间执行大量 RVV 指令，使推理线程的 VS 状态在较长时间内保持 Dirty。Dirty 状态持续时间的增加本身并不会直接提高保存次数，真正影响保存频率的是并发场景下的核心竞争。当可运行的推理线程数量接近或超过可用核心数时，公平调度器需要在线程之间轮转；若被换出的线程处于 Dirty 状态，切换路径便会触发相应的向量上下文保存操作。因此，并发竞争会提高满足 Dirty-V 保存条件的调度事件频率，并使向量上下文管理开销逐步累积。

在空间维度上，公平调度器允许线程根据负载均衡结果在处理器核心之间迁移，且不会依据线程当前是否处于 Dirty V 状态进行区别处理。llama-server 的推理线程与网络处理、请求解析及系统后台任务共享核心资源时，外部负载可能增加推理线程被抢占的概率；即使排除了非推理线程的干扰，当多个推理任务共享有限核心时，推理线程之间仍可能发生频繁轮转。与此同时，跨核迁移还可能破坏线程既有的缓存局部性，进一步增加执行过程中的不确定性。

上述时间与空间因素共同影响 V 状态相关事件的发生频率及其调度开销。下文将结合不同模型规模和并发配置，对二者之间的关系进行量化分析。

2.1.2 量化结果

本项目在指定的 RVV 硬件平台上，分别对 1B 与 3B 量化模型在表列并发配置下进行基线测试，记录吞吐率、首 token 延迟、Prompt 处理耗时、同一时间窗口内的 V 状态保存与恢复条件事件计数，以及不同事件类型对应的切换路径耗时。测试结果见表 2.2 和表 2.3。

表 2.2 不同模型与并发配置下的推理性能及 V 状态切换开销

模型	并发	Wall (s)	TPS	Avg TTFT/ms	P95 TTFT/ms	save 切换总耗时/ms
1B	4	1081	0.418	12588	30892	36309
1B	8	1596	0.471	5303	14805	44399
1B	16	982	0.309	49596	72635	32378
3B	8	2483	0.202	39464	107899	84403

表 2.3 有无 V 状态保存的上下文切换耗时对比

模型	并发	save/次	restore/次	V-save/ns	无 save/ns	差值/us	倍数
1B	4	1,018,160	1,160,909	35661	16584	19.1	2.15×
1B	8	1,344,788	1,682,812	33016	15253	17.8	2.16×
1B	16	931,795	1,084,967	34749	14324	20.4	2.43×
3B	8	2,295,887	2,677,651	36763	15181	21.6	2.42×

对于 1B 模型，并发数由 4 增至 8 时，save 条件事件计数从约 101.8 万增至约 134.5 万；并发数进一步增至 16 后，该计数反而降至约 93.2 万，restore 条件事件也呈现相同趋势。性能指标的变化并不与事件计数完全同步：并发 4 至并发 8 区间内，TPS 略有提高，TTFT 均值和 P95 TTFT 均有所下降；并发增至 16 后，TPS 降低，而 TTFT 均值和 P95 TTFT 显著上升。该结果表明，当并发程度超过可用计算资源的承载范围后，系统调度行为可能发生变化，事件计数与延迟指标不再随并发数同步变化。现有数据尚不足以对这一现象作出确定归因，仍需结合相应配置下的核心占用和线程分布进一步分析。

3B 模型当前仅设置并发 8 一档。与同一并发配置下的 1B 模型相比，3B 模型的 save 条件事件计数约为 229.6 万，restore 条件事件计数约为 267.8 万，TTFT 均值和 P95 TTFT 也相对更高。这一结果说明，在当前测试条件下，模型规模可能同时影响 V 状态条件事件计数和请求延迟；由于 3B 模型仅包含一个并发档位，相关趋势仍需通过更多对照实验进一步验证。

表 2.3 显示，被归类为满足 V 状态保存条件的上下文切换，其平均路径耗时均高于未满足该条件的切换。四组配置下的耗时倍数分布在 2.15 至 2.43 之间，整体波动较小，表明 V 状态相关处理对单次切换路径耗时的影响在当前模型规模和并发配置下具有一定一致性。具体而言，1B 模型三个并发档位的耗时倍数分别为 2.15、2.16 和 2.43。尽管并发 16 档的事件计数有所回落，其耗时倍数却为三个档位中的最高值，说明并发程度更直接地影响事件触发频率，而单次切换的附加耗时还可能受到其他运行因素影响。

为进一步考察线程在空间维度上的迁移行为，本项目将 llama-server 的推理计算线程数设置为 4，并统计各计算线程在不同 CPU 核心上的 V 状态保存条件事件分布，结果如表 2.4 所示。

表 2.4 同一进程各计算线程在不同核心上的 V 状态保存分布

模型规模	并发请求数	线程	Save 总数	涉及核心数	迁移占比 (%)
1B	2	线程 1	903	3	2.0
1B	2	线程 2	738	1	0.0
1B	2	线程 3	555	1	0.0
1B	2	线程 4	245	1	0.0
1B	4	线程 1	1336	1	0.0
1B	4	线程 2	1331	1	0.0
1B	4	线程 3	805	1	0.0
1B	4	线程 4	770	1	0.0
1B	8	线程 1	9164	8	62.7
1B	8	线程 2	7990	8	70.6
1B	8	线程 3	7954	8	70.4
1B	8	线程 4	6895	8	65.0
3B	2	线程 1	5657	2	0.7
3B	2	线程 2	2591	4	5.6
3B	2	线程 3	2561	1	0.0
3B	2	线程 4	2330	1	0.0
3B	4	线程 1	7848	3	13.3
3B	4	线程 2	4247	3	0.1
3B	4	线程 3	3508	5	33.9
3B	4	线程 4	3117	4	37.2
3B	8	线程 1	7009	6	50.0
3B	8	线程 2	5928	2	43.9
3B	8	线程 3	4968	4	1.2
3B	8	线程 4	4699	2	0.1

数据表明，计算线程存在不同程度的核间迁移，且迁移程度并不随并发请求数严格单调变化。对于 1B 模型，并发 2 和并发 4 时多数线程的迁移占比较低；并发增至 8 后，四个线程涉及的核心数均达到 8，迁移占比升至约 63% 至 71%。3B 模型在较高并发条件下也表现出更广泛的核心分布，但不同线程之间的迁移程度差异较大。

需要说明的是，表 2.4 中的迁移占比只刻画计算线程的保存事件是否分布到多个核心，不能单独等同于整体调度开销。总耗时、V-switch 次数和单次切换延迟还会同时受到可运行线程数量、核心竞争强度与 CPU 放置范围的影响。因此，后续实验将“是否超订”和“是否绑定”作为两个相互独立的控制变量，通过四组组合对照区分线程竞争与核间迁移各自带来的影响，避免将超订造成的调度轮转增加误判为绑定策略的直接结果。

为进一步区分线程竞争与 CPU 绑定对调度切换开销的影响，本项目设置了超订与不超订、绑定与不绑定两组控制变量，共形成四种对照配置。其中，超订配置启用 8 个工作线程，绑定配置将工作线程限制在 3 个 CPU 内；每组配置均处理 20 个推理请求。实验结果见表 2.5。

表 2.5 超订与 CPU 绑定条件下涉及 V 上下文任务的调度切换路径统计

配置	20 请求 总耗时 (s)	V-switch 次数占比	V-time 耗时占比	V 平均 延迟 (ns)	非 V 平均 延迟 (ns)
不超订 + 不绑定	964.2	12.0%	34.5%	13861	3600
不超订 + 绑定	364.8	1.39%	2.54%	6605	3584
超订 + 不绑定	1863.9	39.9%	64.6%	12114	4408
超订 + 绑定	1291.5	38.9%	58.7%	7062	3164

结果表明，线程超订会明显提高 V-switch 的次数占比和耗时占比。在不绑定 CPU 的条件下，超订使 V-switch 次数占比由 12.0% 增至 39.9%，耗时占比由 34.5% 增至 64.6%；四组配置中，V 类切换的平均延迟约为非 V 类切换的 1.84–3.85 倍。在超订条件下，CPU 绑定对 V-switch 次数占比的影响较小，但其平均延迟由 12114 ns 降至 7062 ns，20 个请求的总耗时也由 1863.9 s 降至 1291.5 s。就当前配置而言，CPU 绑定的主要收益体现为降低切换路径的平均耗时，而非显著改变 V-switch 的相对占比。

2.1.3 优化策略

基于上述问题分析，本项目从时间和空间两个维度制定优化策略。在时间维度上，通过调整调度策略和推理时间片，减少计算线程在关键计算区间内被频繁抢占和轮转的概率；在空间维度上，通过 CPU 亲和性与资源隔离约束线程的可运行核心范围，降低推理线程的跨核迁移，并减少网络处理线程、控制线程及系统后台任务对计算核心的干扰。

在核心放置与资源隔离方面，taskset 通过 CPU 亲和性将服务进程及其子线程限制在指定核心范围内，但不会约束系统中的其他进程；cgroup cpuset 对控制组内全部线程施加统一的核心范围约束；partition-root 在此基础上建立独立调度分区，阻止其他进程迁入目标核心；systemd-scope 则利用系统作用域和资源切片，将系统进程与推理进程分别限制在不同的核心集合中。

在调度策略方面，SCHED_BATCH 在绑核基础上采用批处理调度，降低推理任务因交互性调度而被抢占的倾向；SCHED_FIFO 则将推理进程设置为高优先级先进先出实时任务，使其在目标核心上尽量避免被普通任务抢占。两种方案均旨在降低上下文切换频率，但采用实时策略时，还需同步评估其对系统公平性、交互响应和可用性的影响。

在可编程调度方面，sched_ext 允许 BPF 程序介入底层调度决策，为 llama-server 推理线程设置明确的核心放置偏好。项目将推理线程优先调度至 CPU 0–3 的 AI 计算核心，并将非推理线程引导至其他核心；同时采用 30 ms 的自定义推理时间片，以减少频繁抢占引起的 V 状态相关切换开销。

2.2 技术路线

本项目遵循分阶段、模块化的技术路线，依次完成实验平台构建、运行状态观测、调度策略设计和自动化评测，逐步形成面向 RISC-V 平台的 LLM 并发推理性能分析框架。整体技术路线如图 2.1 所示。

上述路线遵循“先建立可重复基线，再识别开销来源，随后实施调度干预，最后验证优化效果”的基本思路。平台构建阶段为后续实验提供一致的软硬件环境；观测阶段用于建立推理性能与内核调度行为之间的对应关系；优化阶段依据量化结果选择干预对象；评测阶段则在统一参数下检验不同方案的实际影响。各阶段共享模型配置、请求参数、CPU 范围和实验标识，从而避免因环境差异或采集窗口不一致而对性能变化作出错误归因。

技术实现首先在真实 RVV 硬件上搭建推理实验平台，统一开发板、操作系统、网络和编译环境，为量化模型部署、多路并发请求及内核观测工具的稳定运行提供基础。在此基础上，项目实现低侵入式 V 状态追踪探针，从进程、线程和 CPU 三个维度记录满足 V 状态保存与恢复条件的代理事件并进行归因；同时构建并发推理量化工具，采集吞吐率、首 token 延迟和端到端延迟等性能指标，使推理性能数据与 V 状态观测结果能够在同一实验窗口内对应分析。

观测环节同时覆盖应用层与内核层。应用层记录请求完成情况、生成 token 数、吞吐率以及不同口径的延迟指标，用于描述推理服务的外部性能；内核层记录目标线程的调度切换、CPU 分布和 V 状态条件事件，用于刻画性能变化对应的系统行为。两类数据按照同一轮实验的时间窗口和配置编号进行关联，并分别保留请求明细、聚合结果和原始观测输出。这种设计并不将事件相关性直接等同于因果关系，而是为分析“性能变化是否伴随调度行为改变”提供可复核的证据。

调度优化阶段围绕 CPU 放置、资源隔离和调度优先级，构建 taskset、cpuset、partition-root、systemd-scope、taskset-chrt-b 和 taskset-chrt-f 六种对照配置，并在支持 sched_ext 的内核上实现自定义调度器。该调度器将 llama.cpp 推理线程限制在 SpacemiT K1 的 CPU 0-3 AI 计算核心上，采用 30 ms 的自定义推理时间片，并将普通任务引导至 CPU 4-7，以减少不必要的调度轮转、跨核迁移及相关 V 状态条件事件。

各调度方案按照干预程度由浅入深组织。taskset 和 cpuset 主要约束任务的可运行核心范围，partition-root 与 systemd-scope 进一步强化资源分区和系统任务隔离，SCHED_BATCH 与 SCHED_FIFO 则直接改变目标任务的调度属性。这六种配置基于现有 Linux 机制构建，适合在统一环境下比较部署复杂度、隔离能力和运行风险。sched_ext 原型进一步介入任务识别、选核、入队和时间片分配等决策环节，用于验证专用调度逻辑的可行性。由于两类实



图 2.1 技术路线架构

验所依赖的内核环境和验证目标并不完全相同，其结果分别用于性能比较和机制验证，不进行脱离实验条件的直接排名。

最后，自动化评测工具将服务管理、策略部署、并发请求、性能采集、V 状态追踪和环境清理串联为统一流程。项目首先通过模块测试验证各组件的独立功能，再在一致的环境与参数配置下，对参考组和各优化方案执行多轮集成试验，统计均值、分位数及波动情况，并结合 V 状态条件事件的变化评估不同调度方案对吞吐率和延迟的影响及其稳定性。

为保证实验过程可复现，自动化流程在每轮测试前统一检查模型、服务参数、调度配置和观测工具状态，测试结束后完成进程终止、策略撤销和临时资源清理。实验产生的配置、日志、请求明细和汇总数据按照统一规则归档，使单轮结果能够追溯至具体模型、并

发度、CPU 放置方式和调度策略。由此，整个技术路线形成了从机理分析、量化观测、策略实施到实验验证的闭环，为后续章节中的系统实现和结果分析提供一致的证据基础。

2.3 代码原创声明

本项目软件源代码中第三方库引用情况说明如下：

2.3.1 V 状态追踪脚本

脚本路径为 `src/tracker/vstate_trace.bt`。该脚本为完全原创。基于 `bpfftrace` 的 `tracepoint` 机制，通过挂载 `sched: sched_switch` 事件实时采集 `riscv_vstate_save` 调用频次。具体的 `bpfftrace` 脚本实现均为独立编写，未引用任何第三方 `bpfftrace` 脚本代码。

2.3.2 性能评测脚本

脚本路径为 `src/benchmark/benchmark_llama.py`。该脚本为完全原创，依赖 Python 第三方库 `requests`（HTTP 通信）和 `numpy`（统计计算），这些库仅作为通用依赖工具使用。脚本核心逻辑（包括多轮次并发策略、`VStateOverhead` 类对 `bpfftrace` 的封装、远程 SSH 交互等）均为独立编写。

2.3.3 自动化集成测试框架

脚本路径为 `ctest/bench_runner.py`。该脚本为完全原创，依赖 Python 第三方库 `paramiko`（SSH 通信）、`requests`（HTTP 请求）、`numpy`（统计计算）。脚本核心设计（包括 6 种调度方法的统一抽象、per-CPU V-saves 泄漏检测、双 CSV 输出机制等）均为独立编写。

2.3.4 llama.cpp 推理框架

`llama.cpp` 作为底层 LLM 推理引擎被直接编译使用，未对其任何源文件进行修改。编译时通过 `cmake` 选项启用 RVV 1.0 向量化后端（`GGML_RVV=ON`），使用 GCC 14 工具链确保 RVV 编译支持。

总结：本项目所有脚本代码均为原创，未引用任何第三方源代码片段。使用的第三方组件（`llama.cpp`、`bpfftrace`、`requests`、`paramiko`、`numpy`）均以未修改的二进制或库形式通过标准接口调用，不涉及代码级别的引用或改编。

第 3 章 项目介绍

本章承接第二章提出的设计思想与技术路线，围绕系统总体架构及核心模块的工程实现展开。首先说明系统的分层结构、模块边界、接口关系与数据流；随后依次介绍 V 状态追踪、量化分析和调度优化模块的实现机制；最后阐述自动化性能评测模块如何组织服务控制、数据采集与结果归档，从而形成完整的实验执行链路。

3.1 总体设计

系统总体设计从分层架构和模块接口两个方面展开。分层架构用于明确用户态工具、内核调度机制与处理器资源之间的职责边界；模块接口则描述观测数据和调度控制指令在各层之间的传递方式，为后续各功能模块的独立实现与协同运行提供统一基础。

3.1.1 系统架构

项目整体架构如图 3.1 所示，由用户层、内核层和 CPU 层构成。

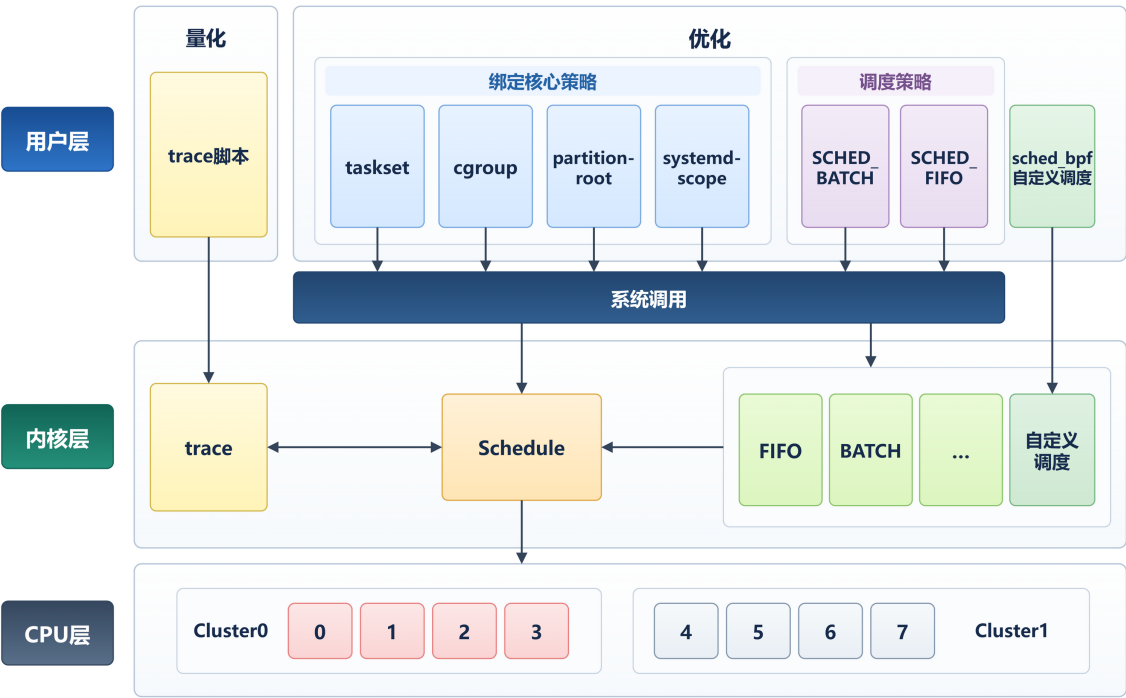


图 3.1 系统架构图

用户层由 V 状态量化和调度优化两部分组成。量化部分通过 Trace 脚本采集调度切换过程中的 V 状态条件事件，并为性能评测模块提供内核侧观测数据。优化部分包含 taskset、cgroup、partition-root 和 systemd-scope 等 CPU 放置与资源隔离配置，SCHD_BATCH、SCHD_FIFO 等调度策略，以及基于 sched_ext BPF 的自定义调度方案。上述配置通过系统调用、控制组接口或 BPF 加载机制作用于内核。

内核层包含 Trace 观测模块、Linux 通用调度器及相关调度类。Trace 模块在调度切换等关键事件处采集线程运行状态，调度器则依据公平调度、实时调度或 sched_ext 自定义策略，对可运行线程执行选核、入队和派发，并将其分配至具体处理器核心。

CPU 层由 Cluster0 和 Cluster1 两组核心构成。其中，Cluster0 包含 CPU 0–3，作为 SpacemiT K1 的 AI 计算核心，主要承载向量密集型推理线程；Cluster1 包含 CPU 4–7，主要运行服务控制线程、系统后台任务及其他通用负载。该分层为后续实施核心绑定、资源隔离和自定义选核策略提供了明确的硬件边界。

3.1.2 接口关系

各模块之间的接口可按照数据流和控制流划分为五条主要路径。

Trace 脚本与内核 Trace 模块构成调度行为观测接口。用户层的 bpffrace 脚本挂载调度切换等内核事件，读取相关线程的 V 状态条件信息，并将按照进程、线程和 CPU 分类的统计结果输出至文件。性能评测模块通过 SSH 控制 Trace 脚本的启动与终止，读取实验开始和结束时的累计结果，并以两次读数之差计算单轮试验的净事件计数。

CPU 放置与隔离配置通过内核资源控制接口施加约束。taskset、cgroup、partition-root 和 systemd-scope 等方案分别利用系统调用、cgroup 文件系统或 systemd 接口，限定 llama-server 及其推理线程的可运行 CPU 范围，内核调度器随后在该约束范围内完成任务选核与切换。

调度策略接口负责连接用户层配置与内核调度类。SCHD_BATCH 和 SCHD_FIFO 通过标准调度接口设置推理进程及其线程的调度属性，使目标任务进入相应的批处理调度或实时调度路径。

sched_bpf 加载程序与 sched_ext 自定义调度器构成可编程调度接口。用户层负责加载 BPF 调度程序、配置目标推理进程并读取运行统计；内核侧的 sched_ext 调度器据此识别推理线程，完成选核、任务派发和 30 ms 推理时间片分配。

内核调度器与 CPU 层之间形成最终的任务执行路径。调度器依据线程所属策略及 CPU 约束，将可运行线程派发至 Cluster0 或 Cluster1 中的具体核心；Trace 模块同步记录执行过程中的调度切换和 CPU 归因信息。由此，用户层的策略配置、内核层的调度决策以及 CPU 层的实际运行结果能够通过观测数据形成闭环。

3.2 Trace 脚本模块

Trace 脚本模块以 `bpfttrace` 程序的形式运行于开发板后台，负责观测内核调度路径中与 V 状态保存和恢复相关的事件。该模块包含两个脚本：`vstate_trace.bt` 从系统和进程两个维度汇总事件；`vstate_trace_cpu.bt` 则在此基础上增加 per-CPU 和 tid×CPU 两个统计维度，用于刻画 V 状态保存事件在不同处理器核心及线程之间的分布。

3.2.1 追踪原理

Linux 内核调度器将线程切出处理器时，仅在线程的 VS 状态为 Dirty 的情况下，才会在上下文切换路径中保存向量寄存器组。该判断位于内核调度主函数 `__schedule()` 调用 `context_switch()` 的执行过程中，随后经由 `switch_to()` 宏进入 `__switch_to_vector()`，最终由 `riscv_v_vstate_save()` 完成 V 状态保存。

然而，`riscv_v_vstate_save()` 在内核中被声明为 `static inline`，通常不会生成可供 kprobe 稳定定位的独立符号；其调用路径中的 `__switch_to()` 又被标记为不可追踪函数，因而同样无法直接挂载探针。受上述条件限制，脚本难以在寄存器保存动作的实际执行位置进行插桩。

基于上述限制，Trace 脚本选择挂载 `tracepoint:sched:sched_switch`，并通过间接判定方式观测 V 状态保存事件。该 tracepoint 由 `trace_sched_switch()` 触发，在 `__schedule()` 中的调用位置早于 `context_switch()`，因此其触发时刻也早于 `riscv_v_vstate_save()` 的实际执行时刻。

当该事件触发时，脚本首先获取被切换出线程的 `task_struct` 指针，再依据内核栈布局计算该线程 `pt_regs` 结构体的内存地址，读取其中保存的 `sstatus` 值并提取 VS 状态位。此处获得的 `sstatus` 是线程上一次因系统调用、外部中断或时钟中断陷入内核时留存在内核栈中的状态快照。由于线程进入内核后至实际发生调度切换前，VS 状态通常不会再次变化，因此该快照可作为判断本次切换是否满足 V 状态保存条件的代理指标。换言之，脚本并不直接记录寄存器保存函数的执行，而是在保存动作发生前，根据线程陷入内核时的 VS 状态推断其是否满足 Dirty 条件，并据此完成间接计数。

V 状态恢复事件采用与保存事件相对应的判定逻辑：脚本在 `sched_switch` 触发时读取被切换入线程此前缓存的状态，并结合进程启动时间进行校验，以降低 PID 重用对统计结果的干扰。

计时方案以 `sched_switch` 作为起点 A，以 `kprobe:finish_task_switch*` 作为终点 B。两点之间的时间区间覆盖 `switch_to` 宏的主要执行过程，其中既包括保存条件判断，也可能包括实际的 V 状态保存操作 `__riscv_v_vstate_save`。在起点处，脚本依据前述代理判定记录 `prev` 线程的 VS 状态是否为 Dirty，并将上下文切换划分为“满足 V 状态保存条件”和“不满足

V 状态保存条件”两类；到达终点后，再分别使用 `stats()` 聚合两类切换的纳秒级耗时。在其他调度开销近似同分布的假设下，两组平均耗时之差可用于估计 V 状态保存引入的附加延迟。该结果反映的是基于分组差分得到的近似开销，而非对单次寄存器保存过程的直接计时。

3.2.2 数据格式

脚本运行期间每 10 秒输出一组累计统计快照，其中包含 `saves_total` 和 `restores_total` 两项计数。

脚本终止时输出完整的统计汇总，包括 `saves_total`、`restores_total`、`saves[comm,pid]`、`restores[comm,pid]`、`cpu_saves[cpu]`、`cpu_restores[cpu]` 和 `cpu_saves_by_proc` 等统计项。上述计数均为本次追踪会话启动以来的累计值，可作为量化分析模块计算试验窗口增量的原始数据。

3.3 量化分析模块

量化分析模块封装于基准测试脚本 `benchmark_llama.py` 的 `VStateOverhead` 类中，负责控制 `bpfttrace` 追踪进程的启动与停止，并采集试验窗口内的 V 状态保存和恢复事件增量。在此基础上，模块将观测结果与具体试验配置关联，用于分析 V 状态切换开销随模型规模和并发请求数变化的规律。

3.3.1 工作流程

量化模块运行于 x86 上位测试机，并通过 SSH 与 RISC-V 开发板建立控制和数据传输通道。完整采集流程由启动、采集和停止三个阶段构成，三个阶段共享同一组累计计数基线，从而保证连续试验之间的数据能够正确分离。

1. 启动推理服务并建立采集基线。运行量化模块前，首先通过 `bench_runner` 或手动方式在 RISC-V 开发板上启动 `llama-server`。以 1B 模型和 8 路并发配置为例，启动命令如下：

```
./llama-server -m <model> --port 8080 -t 4 -c 8192 --parallel 8
```

服务启动后，量化模块以 2 秒为间隔轮询健康检查端点：

```
curl.exe --silent --show-error http://IP:PORT/health
```

确认服务就绪后，模块通过 SSH 在开发板后台启动 `bpfftrace`，并将追踪结果重定向至 `vstate.txt`。待输出文件生成首个周期性统计快照后，读取 `saves_total` 和 `restores_total`，并将其记录为本次测试会话的初始基线：

```
nohup stdbuf -oL bpfftrace /root/vstate_trace_cpu.bt -o /tmp/vstate.txt > /tmp/
rvv-benchmark-vstate.log 2>&1 & \
echo \${!} > /tmp/rvv-benchmark-vstate.pid"
```

2. 采集试验窗口内的增量。模块通过 SSH 从输出文件中提取最新的 `@saves_total`、`@restores_total`，以及与 `llama-server` 相关的进程归因记录，并据此汇总推理进程对应的保存事件数 `llama_save`：

```
grep -E '@saves_total:|@restores_total:|\
@saves\[.*llama-server|@restores\[.*llama-server' \
/tmp/vstate.txt
```

每轮试验结束后，`run_trial()` 使用当前累计读数减去上一轮基线，得到该轮试验窗口内的事件净增量；随后以当前读数更新基线，供下一轮试验继续使用。该差分方式能够避免不同试验轮次的累计计数相互混淆。

3. 停止采集并读取终值。全部试验结束后，模块通过 SSH 终止 `bpfftrace` 进程，触发脚本的 `END` 处理逻辑并输出最终统计结果，随后调用 `collect()` 读取和返回追踪终值：

```
pid=\$(cat /tmp/rvv-benchmark-vstate.pid); \
kill \${pid} 2>/dev/null || true; \
sleep 2; \
rm -f /tmp/rvv-benchmark-vstate.pid
```

3.3.2 量化指标

量化分析模块输出三类指标：全系统 V 状态保存次数 `save_count` 和恢复次数 `restore_count`，用于描述试验窗口内的总体 V 状态切换活动；`llama-server` 进程对应的保存次数 `llama_save` 和恢复次数 `llama_restore`，用于区分推理负载与其他系统任务产生的事件；各 CPU 核心上的 V 状态保存事件分布 `per-CPU saves`，则用于观察事件在不同处理器核心上的集中程度，并辅助分析线程迁移和任务放置行为。需要说明的是，`per-CPU saves` 反映的是保存事件的空间分布，不能直接等同于线程迁移次数。

3.4 优化模块

3.4.1 粗粒度优化方案

优化模块包含粗粒度资源配置和 `sched_ext` 自定义调度两类策略。粗粒度方案从 CPU 亲和性、资源隔离和调度策略三个维度约束推理任务的运行环境，以降低非推理任务干扰及频繁调度切换带来的开销，并已统一接入自动化评测流程。自定义调度方案则进一步介入任务识别、核心选择、队列派发和时间片分配等调度环节，使内核能够依据推理线程角色和预设核心分区实施更有针对性的调度控制。

CPU 亲和性绑定 (taskset): `taskset` 通过 `sched_setaffinity` 系统调用设置进程的 CPU 亲和性掩码，使内核调度器只能在掩码允许的核心范围内放置该进程及其线程。该方法仅约束推理进程的可运行核心，不限制系统中其他任务进入同一核心集合，因此属于单向的任务放置约束。

以正式评测采用的 CPU 4-7 为例，启动命令如下：

```
taskset -c 4-7 llama-server -m <model> -t 4 -tb 4 --parallel 4 -c 2048 --cont-batching
--port 8080
```

服务启动后，可逐一读取各线程的 CPU 亲和性范围，以验证配置是否生效：

```
pid=$(pidof llama-server)
for t in /proc/$pid/task/*; do
    grep Cpus_allowed_list $t/status
done
```

若配置正确，全部线程的 `Cpus_allowed_list` 均应为 4-7。

cgroup cpuset 隔离: 该方案基于 `cgroup v2` 的 `cpuset` 控制器建立独立控制组，将目标核心写入 `cpuset.cpus`，并将推理进程标识符 (PID) 写入 `cgroup.procs`。与 `taskset` 相比，`cpuset` 能够以控制组为单位统一管理进程及其线程的可运行核心范围，便于在服务运行期间持续维护资源约束。

配置命令示例如下：

```
# cgroup cpuset
echo "+cpuset" > /sys/fs/cgroup/cgroup.subtree_control
mkdir -p /sys/fs/cgroup/llm
echo "4-7" > /sys/fs/cgroup/llm/cpuset.cpus
echo "0" > /sys/fs/cgroup/llm/cpuset.mems
```

```
# 正常启动server后, 获取当前server的PID
echo <PID> > /sys/fs/cgroup/llm/cgroup.procs # 将serverPID加入控制组
```

配置完成后, 可通过以下命令核对设定值、生效范围及控制组中的进程数量:

```
cat /sys/fs/cgroup/llm/cpuset.cpus          # 期望 4-7
cat /sys/fs/cgroup/llm/cpuset.cpus.effective # 期望 4-7
cat /sys/fs/cgroup/llm/cgroup.procs | wc -l  # 期望 >0
```

试验结束后, 使用以下命令撤销控制组配置:

```
echo member > /sys/fs/cgroup/llm/cpuset.cpus.partition 2>/dev/null
rmdir /sys/fs/cgroup/llm
echo "-cpuset" > /sys/fs/cgroup/cgroup.subtree_control
```

partition-root 独占隔离: 该方案在 cpuset 约束的基础上, 将 cpuset.cpus.partition 设置为 root, 使目标核心集合形成独立的调度分区。与仅限制推理进程运行范围的单向约束相比, 独立分区能够进一步降低分区外任务进入目标核心的可能性, 从而增强推理负载与系统负载之间的双向隔离。

配置命令如下:

```
echo "+cpuset" > /sys/fs/cgroup/cgroup.subtree_control
mkdir -p /sys/fs/cgroup/llm
echo "4-7" > /sys/fs/cgroup/llm/cpuset.cpus
echo "0" > /sys/fs/cgroup/llm/cpuset.mems
echo "root" > /sys/fs/cgroup/llm/cpuset.cpus.partition
# 正常启动server后, 获取当前server的PID
echo <PID> > /sys/fs/cgroup/llm/cgroup.procs
```

可通过以下命令检查调度分区状态及实际生效的 CPU 范围:

```
cat /sys/fs/cgroup/llm/cpuset.cpus.partition # 期望 root
cat /sys/fs/cgroup/llm/cpuset.cpus.effective # 期望 4-7
```

试验结束后的清理命令如下:

```
echo 0 > /sys/fs/cgroup/llm/cgroup.procs 2>/dev/null
echo member > /sys/fs/cgroup/llm/cpuset.cpus.partition 2>/dev/null
```

```
rmdir /sys/fs/cgroup/llm
echo "-cpuset" > /sys/fs/cgroup/cgroup.subtree_control
```

systemd-scope 系统级隔离：该方案通过 `systemctl set-property` 分别设置 `init.scope`、`system.slice` 和 `user.slice` 的 `AllowedCPUs`，将系统服务和用户任务限制在非目标核心；同时使用 `systemd-run --scope` 将推理进程置于独立的 `llm.slice`，并为其指定目标核心范围。由此，核心分配策略由 `systemd` 统一映射到 `cgroup` 层，能够在不直接操作控制组文件的情况下实现系统任务与推理任务的资源分离。

配置命令如下：

```
systemctl set-property --runtime init.scope    AllowedCPUs=0-3
systemctl set-property --runtime system.slice AllowedCPUs=0-3
systemctl set-property --runtime user.slice   AllowedCPUs=0-3
systemctl set-property --runtime llm.slice    AllowedCPUs=4-7
systemd-run --scope --unit=llm -p Slice=llm.slice llama-server -m <model> -t 4 -tb 4
--parallel 4 -c 2048 --cont-batching --port 8080
```

配置完成后，可分别查询各作用域和资源切片的 `AllowedCPUs` 属性：

```
systemctl show init.scope    -p AllowedCPUs # 期望 0-3
systemctl show system.slice -p AllowedCPUs # 期望 0-3
systemctl show user.slice   -p AllowedCPUs # 期望 0-3
systemctl show llm.slice    -p AllowedCPUs # 期望 4-7
```

试验结束后，使用以下命令恢复 `systemd` 的运行属性：

```
systemctl set-property --runtime init.scope    AllowedCPUs=
systemctl set-property --runtime system.slice AllowedCPUs=
systemctl set-property --runtime user.slice   AllowedCPUs=
systemctl set-property --runtime llm.slice    AllowedCPUs=
systemctl stop llm.scope 2>/dev/null
systemctl reset-failed 2>/dev/null
```

SCHED_BATCH 调度策略：该方案在 `taskset` 绑核基础上使用 `chrt -b 0`，将推理进程设置为 `SCHED_BATCH` 调度策略。`SCHED_BATCH` 面向非交互式、计算密集型任务，会降低任务在唤醒抢占方面的交互性偏好，使其更适合持续执行的批处理负载；但该策略仍属于公平调度类别，并不提供实时优先级或严格的抢占保证。

启动命令如下：

```
taskset -c 4-7 chrt -b 0 llama-server -m <model> -t 4 -tb 4 --parallel 4 -c 2048 --  
cont-batching --port 8080
```

服务启动后，可查询目标进程的调度策略和优先级：

```
chrt -p $(pidof llama-server)  
# 期望: Scheduling policy: SCHED_BATCH, Priority: 0
```

SCHED_FIFO 实时调度策略：该方案在 taskset 绑核基础上使用 chrt -f 90，将推理进程设置为优先级 90 的 SCHED_FIFO 实时任务。运行时，目标进程通常不会被普通公平调度任务抢占，因而有助于减少上下文切换和 V 状态相关事件。与此同时，高优先级实时任务可能延迟同一核心上的其他任务，甚至在缺少运行时间约束时造成资源饥饿，因此该方案主要用于受控实验环境，不宜在通用系统中直接部署。

启动命令如下：

```
taskset -c 4-7 chrt -f 90 llama-server -m <model> -t 4 -tb 4 --parallel 4 -c 2048 --  
cont-batching --port 8080
```

服务启动后，可通过以下命令核对调度类别和实时优先级：

```
chrt -p $(pidof llama-server)  
# 期望: Scheduling policy: SCHED_FIFO, Priority: 90
```

3.4.2 sched_ext 自定义调度

3.4.2.1 核心痛点

在面向 RISC-V Vector 扩展的大语言模型推理场景中，不同类型的线程对处理器资源具有显著不同的需求。向量计算线程需要持续占用具备目标加速能力的计算核心，而网络服务、请求管理和系统后台任务更适合运行在通用核心域。针对本项目采用的 8 核运行环境，系统将 CPU 0-3 作为 AI 计算核心域，将 CPU 4-7 作为通用核心域。若调度器无法识别线程角色并维持上述资源分区，高并发推理过程中便容易出现计算资源争用和频繁迁移等问题。

Linux 原生 CFS/EEVDF 调度器主要依据任务可运行状态、虚拟运行时间和 CPU 负载实施公平调度，并不直接理解 llama.cpp 线程角色及向量计算上下文的保存代价。在多线程

并行推理过程中，计算密集型线程可能离开预定的 AI 核心域；若线程进一步受到系统任务抢占或频繁发生跨核迁移，其 Cache 工作集将难以稳定保留，并可能增加 Dirty-V 条件下的向量状态保存开销，最终表现为吞吐下降和首 token 延迟波动。

与此同时，llama-server 的主线程主要承担网络 I/O、请求管理和任务分发，向量化计算则主要由工作线程执行。若主线程、系统后台任务与推理工作线程长期混合运行，通用负载会占用原本用于向量计算的处理器时间，并增加工作线程被切出的概率。因此，自定义调度不仅需要约束计算线程的运行核心，还需要区分进程主线程与工作线程，使不同角色的任务进入相应的核心域和派发路径。

3.4.2.2 调度方案设计目标

针对上述问题，本模块基于 eBPF sched_ext 框架设计并实现自定义调度器 vec_affine_sched，其核心目标如下：

- 根据目标进程及线程角色识别推理工作线程，将其限制在 CPU 0–3 的 AI 计算核心域，并将进程主线程和普通任务引导至 CPU 4–7 的通用核心域。
- 推理线程唤醒时优先检查其上一次运行的 CPU，在原核心可用时维持原有放置，以降低跨核迁移并尽可能保留 Cache 工作集。
- 为推理线程分配 30 ms 的计算时间片，并在特定唤醒场景下赋予本地队列首部入队标志，以减少计算过程中的非必要调度中断。

3.4.2.3 sched_ext 框架支持与核心工作原理

sched_ext (SCX) 是 Linux 内核提供的可扩展调度框架，允许通过 eBPF 程序实现任务选核、入队和派发等调度逻辑。与直接修改内核调度器相比，该框架能够在保留内核安全检查和异常回退机制的前提下，以动态加载方式验证面向特定负载的调度策略。

使用 sched_ext 前，需要确保目标内核启用以下配置项：

```
CONFIG_BPF=y
CONFIG_SCHED_CLASS_EXT=y
CONFIG_BPF_SYSCALL=y
CONFIG_BPF_JIT=y
CONFIG_DEBUG_INFO_BTTF=y
CONFIG_BPF_JIT_ALWAYS_ON=y
CONFIG_BPF_JIT_DEFAULT_ON=y
```

内核调度子系统通过 `struct sched_ext_ops` 暴露一组回调接口。BPF 程序实现并注册这些回调后，即可在 `sched_ext` 的约束范围内参与任务调度。本项目使用的主要机制如下：

- `DSQ`（Dispatch Queue）用于组织待运行任务。每个 CPU 均具有对应的 `SCX_DSQ_LOCAL` 本地派发队列，BPF 程序还可通过 `scx_bpf_create_dsq()` 创建由多个 CPU 共享的 `DSQ`。
- `select_cpu` 在任务唤醒选核阶段被调用，用于检查候选 CPU 的空闲状态并给出放置建议；返回目标 CPU 后，内核将据此推进后续入队过程。
- `enqueue` 在任务进入可运行队列时被调用，可通过 `scx_bpf_dsq_insert()` 指定任务进入本地 `DSQ` 或共享 `DSQ`，并同步设置执行时间片和入队标志。
- `dispatch` 在 CPU 需要获取下一项任务时被调用。本项目利用该回调将共享 `DSQ` 中的普通任务转移到通用核心的本地队列执行。

3.4.2.4 分层架构设计

`vec_affine_sched` 采用用户态控制程序与内核态 eBPF 调度引擎协同工作的双层架构，并以 BPF Map 作为控制信息和运行状态的交换通道。用户态负责策略装载、参数下发、运行观测与生命周期管理，内核态负责选核、入队和派发等调度热路径。该划分将配置管理和统计汇总保留在用户态，同时使与任务唤醒直接相关的决策在内核中完成，从而兼顾策略可编程性与调度路径的执行效率。整体结构如图 3.2 所示。

用户态 Loader 基于 `libbpf` 实现。程序首先打开并校验 BPF 对象，随后将 `vec_affine_ops` 作为 `struct_ops` 调度器挂载到 `sched_ext`。当用户通过 `-p <TGID>` 显式指定目标进程时，Loader 调用 `bpf_map_update_elem()` 将 `TGID` 写入 `config_map`；未提供该参数时，则由内核态程序在运行过程中识别目标进程。为支持运行状态复核，Loader 将 Per-CPU 类型的 `stats Map` 固定到 `/sys/fs/bpf/vecsched`，并依据 `libbpf_num_possible_cpus()` 返回的 CPU 数量读取各核心计数器，再通过 `bpf_map_lookup_elem()` 聚合并输出选核、入队与派发统计。

BPF Map 数据通道承担双层架构之间的配置传递和状态同步。其中，`config_map` 保存目标 `TGID`，`stats` 汇总各 CPU 上的选核、入队和派发计数，`task_ctx_map` 记录任务级上下文属性。内核态调度引擎挂载于 `sched_ext` 的 `select_cpu`、`enqueue` 和 `dispatch` 等核心 Hook，根据任务身份与预设核心分区选择运行核心和派发队列。由此，系统在控制面、状态观测面和调度执行面之间形成清晰的职责边界。

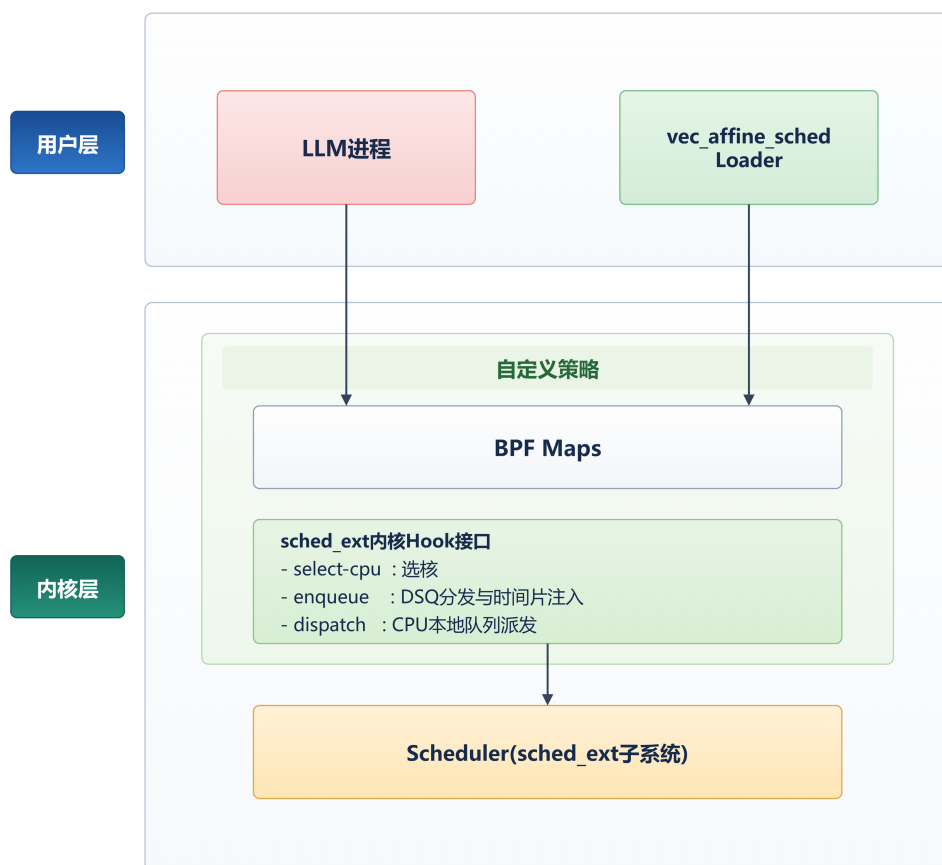


图 3.2 vec_affine_sched 分层架构

在调度策略上，vec_affine_sched 按照任务过滤、目标进程识别、核心域选择和队列派发的顺序处理任务唤醒。任务过滤阶段首先检查 `p->flags&PF_KTHREAD` 和 `p->nr_cpus_allowed==1`。内核线程以及已被限制为单一 CPU 亲和性的任务不再由本策略重新分配，而是交由 `scx_bpf_select_cpu_dfl()` 处理，从而避免覆盖内核任务或用户既有的显式约束。对于其余任务，调度器优先读取 `config_map` 中的 TGID，并与当前任务所属进程进行比对；当映射尚未配置时，再读取 `p->comm` 并匹配 `llama-server`，匹配成功后将 TGID 回写至 `config_map`。对于已识别的目标进程，本实现采用 `p->pid!=p->tgid` 作为工作线程判据，即将非线程组主线程纳入推理线程调度路径，而进程主线程仍按普通任务处理。该判据实现简单且无需修改 `llama-server`，但其识别粒度以线程组关系为基础，尚不能进一步区分进程内部不同类型的辅助线程。

完成身份判定后，调度器依据 Milk-V Jupiter 上预设的核心分区建立两个运行域。在由本策略接管的任务中，LLM 推理工作线程被限制在 CPU 0-3 的 AI 核心域，进程主线程和普通任务则被引导至 CPU 4-7 的通用核心域。该划分一方面使计算线程保持在目标核心范

围内，另一方面减少网络处理、控制逻辑和系统后台任务对推理核心的占用，从调度路径上分离计算负载与通用负载。

推理线程在 AI 核心域内进一步遵循 Cache 亲和性优先的选核规则。若其上一次运行核心 `prev_cpu` 位于 CPU 0-3，且 `scx_bpf_test_and_clear_cpu_idle(prev_cpu)` 确认该核心处于空闲状态，则优先维持原核心放置；原核心繁忙时，调度器再探查 CPU 0-3 中的其他空闲核心。若四个 AI 核心均处于忙碌状态，但 `prev_cpu` 仍属于目标核心域，则任务返回原核心排队，以避免无收益的跨核迁移；仅当原核心不在目标范围内时，才通过 `prev_cpu % 4` 将任务重新映射至 CPU 0-3。该选核顺序在满足核心域约束的同时，尽可能保留线程已有的 Cache 工作集。

选核结果随后由入队策略落实。推理线程获得 `SLICE_INFER_NS(30ms)` 时间片，并进入目标 CPU 对应的 `SCX_DSQ_LOCAL` 本地队列；当任务由睡眠状态被唤醒时，调度器附加 `SCX_ENQ_HEAD` 标志，使其进入本地派发队列头部。普通任务采用 `SLICE_DEFAULT_NS(4ms)` 时间片并进入 `SHARED_DSQ`，由 CPU 4-7 在 `dispatch` 阶段按需拉取。较长的推理时间片用于降低计算批次中途发生调度轮转的频率，本地队列用于维持 CPU 亲和性，而普通任务的共享队列则保留通用核心之间的负载分担能力。

用户态 Loader 还负责调度器的退出与恢复。程序捕获 `SIGINT` 或 `SIGTERM` 信号后，依次调用 `vec_affine_sched_bpf__detach()` 和 `bpf_map__unpin()`，解除 `vec_affine_ops` 挂载并清理固定在 BPF 文件系统中的统计映射。`sched_ext` 调度器退出后，相关任务重新由内核默认公平调度器管理。因此，策略加载、运行观测、主动卸载和恢复过程均由同一控制程序管理，无需修改推理程序本身。

上述组件的调用关系如图 3.3 所示。整个运行过程可以归纳为初始化、运行时调度和卸载恢复三个阶段。

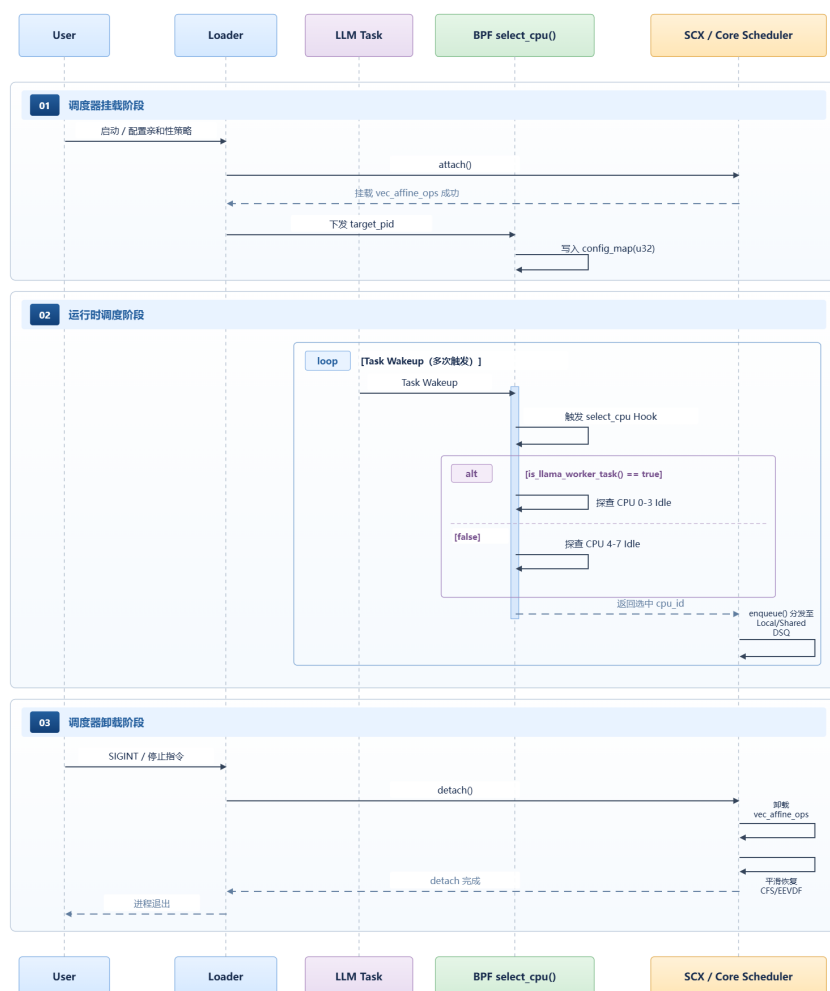


图 3.3 sched_ext 自定义调度流程

初始化阶段，Loader 通过 `vec_affine_sched_bpf__open_and_load()` 完成 BPF 对象的加载与校验，再调用 `vec_affine_sched_bpf__attach()` 将 `vec_affine_ops` 挂载到 `sched_ext`。若用户提供目标 TGID，Loader 同时更新 `config_map`；否则保留内核态动态识别路径。统计 Map 也在此阶段被固定到 BPF 文件系统，供 Loader 和 `bpftool` 等工具在运行期间读取。

运行阶段，由本策略接管的任务在唤醒时进入 `select_cpu`。LLM 推理工作线程在 CPU 0–3 范围内执行亲和性选核，进程主线程和普通任务则被引导至 CPU 4–7 的通用核心域。选核结果返回 `sched_ext` 后，`enqueue` 根据任务类型将其送入本地 DSQ 或共享 DSQ；当通用核心需要补充可运行任务时，`dispatch` 再从共享队列中拉取任务。相关事件同步累计到 `Per-CPU stats` 中，使调度决策能够通过计数器持续观测和复核。

卸载阶段，Loader 在收到终止信号后解除 `vec_affine_ops` 挂载、取消 Map Pin 并释放用户态资源。相关任务随后由内核默认公平调度器继续管理，正在运行的推理服务无需重启即可继续执行。该过程为实验结束和策略切换提供了明确的恢复路径；若 `sched_ext` 检测到调度器异常，则由内核侧安全机制触发退出并恢复默认调度。

3.4.2.5 集成验证与场景边界

`vec_affine_sched` 的功能正确性与运行安全性从核心放置、动态识别、时间片策略和卸载回退四个方面进行验证，具体测试方法及预期现象见表 3.1。这些测试用于确认调度器已进入预定执行路径，不直接替代第四章中的性能评测。

表 3.1 `vec_affine_sched` 功能验证方案

验证项	测试步骤与操作	预期观察结果
核心放置验证	启动 Loader 并运行 llama-server 推理任务，使用 <code>htop</code> 和 <code>mpstat -P ALL 1</code> 观察各 CPU 的负载分布。	推理工作线程主要运行于 CPU 0-3，CPU 4-7 主要承载进程主线程和系统后台任务。
动态识别测试	不输入 <code>-p</code> 参数启动 Loader，随后启动 llama-server 进程。	Loader 日志显示 <code>config_map</code> 已写入 llama-server 的 TGID，且 <code>inference_threads</code> 计数随推理任务运行而增加。
时间片策略验证	在运行大模型推理的同时施加高频 I/O 干扰任务，并记录工作线程的调度统计。	工作线程获得 30 ms 时间片，相关入队计数持续增加，调度行为符合预设策略。
卸载回退测试	在推理持续进行过程中向 Loader 进程发送 SIGINT (Ctrl+C) 信号。	Loader 输出卸载信息， <code>sched_ext</code> 注册项消失；推理服务继续运行，相关任务恢复由内核默认公平调度器管理。

在目标内核已启用 `sched_ext` 及相关 BPF 配置的前提下，该方案无需再次修改或编译内核，也无需改动 llama-server，即可通过 eBPF 结构体操作集动态加载。BPF 字节码在加载阶段接受内核 Verifier 校验，SCX watchdog 则在调度器异常或调度停滞时触发退出和回退。与此同时，选核与 Prev-CPU Idle 检测位于任务唤醒路径中，可在控制额外判断开销的同时维持线程的 L1/L2 Cache 亲和性。

该方案也具有明确的适用边界。当前 `vec_affine_sched` 采用针对 SpacemiT K1 试验环境预设的 8 核分区，即 CPU 0-3 为 AI 计算核心域，CPU 4-7 为通用核心域；迁移至 4 核、16 核或采用不同核心分区的平台时，需要调整内核态常数并重新编译 BPF 程序。30 ms 时间片主要面向 LLM 推理和矩阵运算等计算密集型、长执行周期负载，不适合直接应用于高频 I/O 或微秒级响应服务，否则可能增加交互任务的等待时间。此外，当前工作线程识别依赖

TGID、进程名和线程组关系，对于具有复杂辅助线程结构的推理框架，仍需引入更细粒度的线程角色标识机制。

3.5 性能评测模块

性能评测模块是本项目自动化试验流程的控制中枢，采用 Python 实现并运行于 x86 测试主机。该模块通过 SSH 远程控制 RISC-V 开发板上的服务进程、调度配置与观测工具，同时按照预设顺序遍历测试参数和调度方法，完成请求发送、数据采集、环境清理及结果归档，最终生成结构化的 CSV 试验数据。

模块使用 paramiko 建立 SSH 控制通道，使用 requests 发送 HTTP 请求并解析 SSE 流式响应，使用 numpy 完成延迟分位数等统计计算。各依赖分别承担远程控制、请求采集和数据汇总功能，使试验控制逻辑与被测推理服务相互分离。

性能评测过程中各模块之间的接口关系如图 3.4 所示。

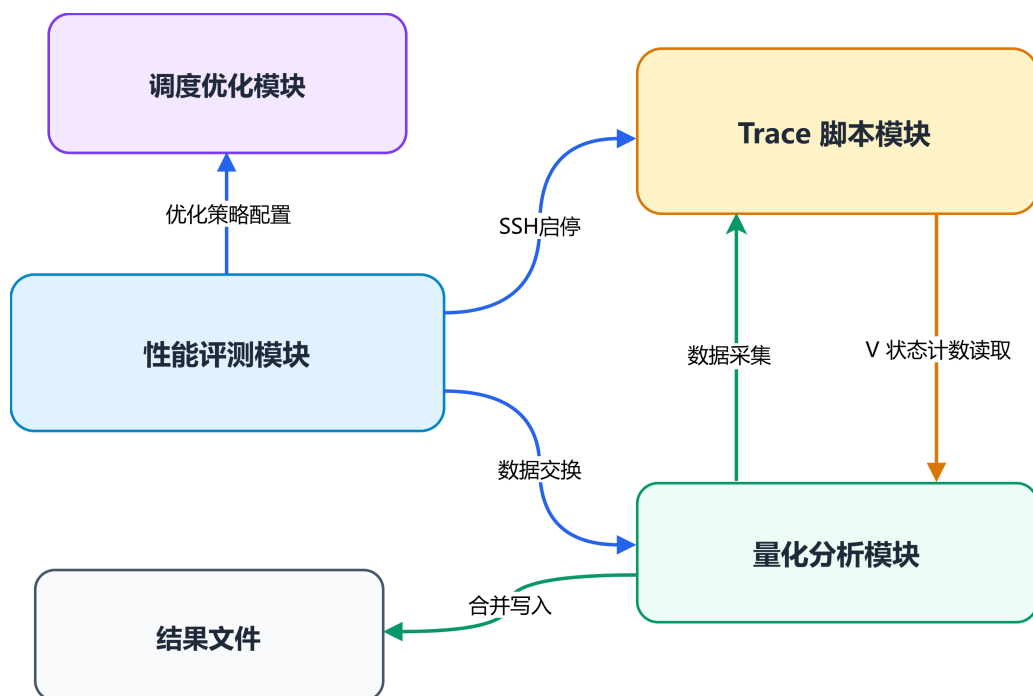


图 3.4 模块接口关系图

3.5.1 模块执行流程

模块的执行主体采用三层嵌套循环组织。外层循环从 GROUPS 列表中读取本轮启用的参数配置，中间层循环遍历 METHODS 列表中的六种调度方法，内层循环执行三次独立重复试验，整体流程如图 3.5 所示。GROUPS 共维护十组候选配置，可根据试验目标选择其

中部分配置执行；第四章正式对比使用六组配置，因此正式结果共包含 $6 \times 6 \times 3 = 108$ 轮试验。

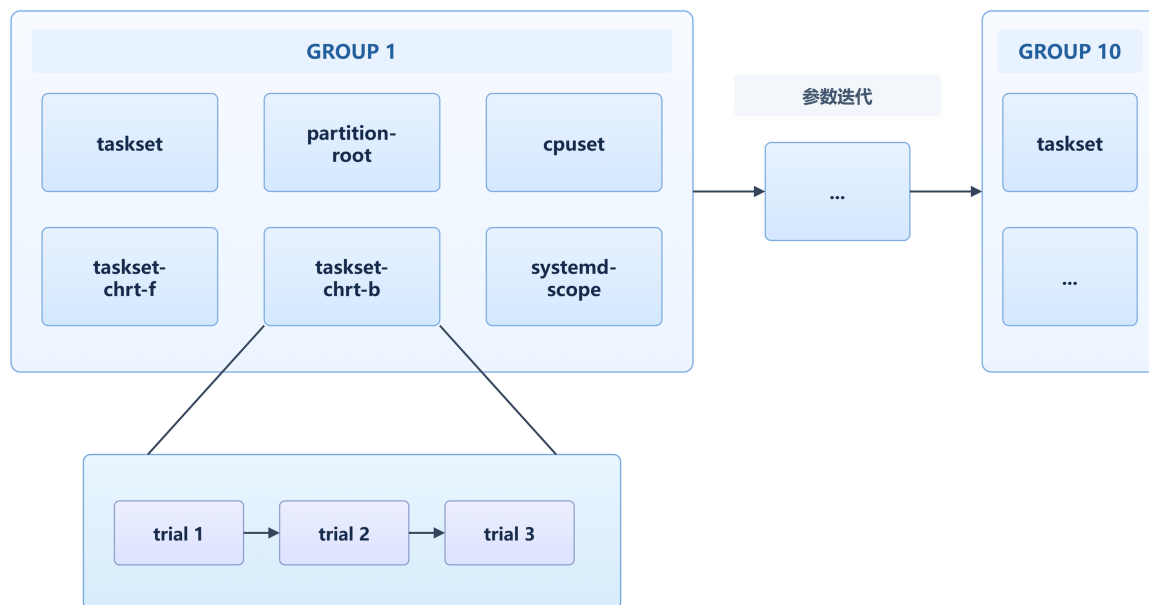


图 3.5 性能评测参数遍历与重复试验流程

每轮试验均按照相同顺序完成环境配置、服务启动、请求执行、数据采集和状态恢复，具体步骤如下：

1. 执行当前调度方法对应的预置脚本，创建所需的 **cgroup** 层次结构，或设置 **systemd** 服务与作用域的 CPU 亲和性参数。
2. 根据当前参数配置和调度方法构造 **llama-server** 启动命令，并通过 **SSH** 在开发板后台启动推理服务。
3. 由测试主机周期性访问健康检查端点（**/health**），直至推理服务返回正常响应。
4. 通过 **SSH** 在开发板后台启动 **bpfftrace**，并加载 **vstate_trace_cpu.bt** 脚本，记录试验开始前的累计计数基线。
5. 由测试主机的请求生成模块使用线程池，以并发或固定时间间隔向推理服务发送 **HTTP** 请求。
6. 等待全部请求完成，解析 **SSE** 流式响应，并记录服务端返回的 **token** 数量与时间信息。

7. 终止 `bpfttrace` 进程，读取最终统计文件，并根据试验前后的累计读数计算本轮 V 状态条件事件增量。
8. 终止本轮 `llama-server` 进程，避免服务状态延续至下一轮试验。
9. 汇总请求记录，计算聚合吞吐量、平均首 token 延迟、P95 首 token 延迟、V 状态条件事件及其 CPU 分布，并分别写入汇总表和明细表。
10. 检查并终止残留的 `llama-server` 和 `bpfttrace` 进程，清除 `cgroup` 目录及 `systemd` 运行时配置，使下一轮试验从一致的系统状态开始。

GROUPS 列表维护的十组模型、线程数和请求规模配置见表 3.2。

表 3.2 测试参数配置

模型	threads	绑核	prompt (token)	output (token)	ctx (token)	parallel
1B	1	1	64	32	1024	2
1B	2	2	64	32	1024	2
3B	4	4	128	64	2048	2
3B	4	4	128	32	2048	4
3B	4	4	64	64	2048	4
3B	4	4	128	64	2048	4
3B	4	4	256	64	2048	4
3B	4	4	128	128	2048	4
3B	4	4	128	64	2048	8
3B	4	4	256	256	4096	4

十组候选参数采用分层控制变量设计。前两组使用 1B 模型，比较单线程、单核与双线程、双核配置下的基础扩展特征；其余配置以 3B 模型、4 个推理线程和 4 个绑定核心为主体，分别调整并行槽位、输入长度、输出长度及上下文窗口，以观察并发压力、预填充负载和解码负载对吞吐、首 token 延迟及 V 状态条件事件的影响。最后一组同时增大输入长度、输出长度和上下文窗口，用于构造长输入与长生成叠加的压力场景。除表中列出的变量外，同一批次试验的软硬件环境、调度方法和执行流程均保持一致，以减少无关因素对方案比较的干扰。

3.5.2 采集指标与输出

模块以单次请求记录为基础，在每轮试验结束后计算性能指标和 V 状态观测指标。

聚合吞吐量 `agg_tps` 由该轮生成的 token 总数除以整轮试验的挂钟时间得到，用于衡量并发请求条件下的整体生成能力。平均首 token 延迟 `avg_ttft_ms` 由该轮全部请求的 TTFT

样本取算术平均值，P95 首 token 延迟 `p95_ttft_ms` 则取同一组样本的第 95 百分位，用于描述多数请求之外的尾部响应情况。

V 状态相关指标包括 `v_saves`、`v_restores`、`target_saves`、`nontarget_saves` 及各 CPU 核心上的事件分布。其中，前两项表示试验窗口内的系统级保存和恢复条件事件增量，后两项用于区分目标推理进程与其他任务产生的保存条件事件。上述指标来自 Trace 脚本的代理判定，用于刻画调度切换行为，不等同于硬件寄存器保存函数的直接执行次数。

每完成一种调度方法的测试，模块分别生成汇总表和明细表两类 CSV 文件。明细表保留请求级延迟、token 数量和试验标识，便于复核单个样本；汇总表按参数配置、调度方法和重复轮次记录聚合指标，用于后续统计分析和方案比较。两类文件共享统一的配置标识，从而能够由汇总结论追溯至对应的原始请求记录。

第 4 章 实验测试

本章围绕系统各功能模块的正确性、调度配置的可复现性以及不同方案在并发推理场景下的运行表现开展实验验证。首先介绍测试平台的软硬件环境、模型配置与实验组织方式，随后通过模块测试验证推理服务、V 状态追踪、量化分析和调度控制等功能能否按照设计独立运行，并通过集成测试考察各模块协同工作时的端到端执行过程。在结果分析中，本文结合吞吐量、首 token 延迟、端到端延迟及 V 状态条件事件等指标，对六种通用调度方法进行统一口径下的比较，同时单独呈现 sched_ext 自定义调度器的功能与性能验证结果。鉴于两类实验所采用的内核环境和参数配置并不完全相同，其结果分别用于评价现有调度机制和验证自定义调度原型，不作脱离实验条件的直接排序。

4.1 测试环境

4.1.1 硬件环境

测试环境由 x86 测试主机和 Milk-V Jupiter 推理服务端组成，两者通过千兆以太网直连。测试主机负责运行自动化评测脚本、生成并发请求、汇总试验数据及执行后续统计分析；推理服务端采用 RISC-V 架构，负责运行 llama.cpp 推理服务、调度策略和 bpftime 观测程序。Milk-V Jupiter 同时通过串口连接至调试主机，用于系统管理、启动过程观察和内核日志采集。该部署方式将负载生成与推理执行分离，避免测试脚本直接占用被测平台的处理器资源，整体连接关系如图 4.1 所示。

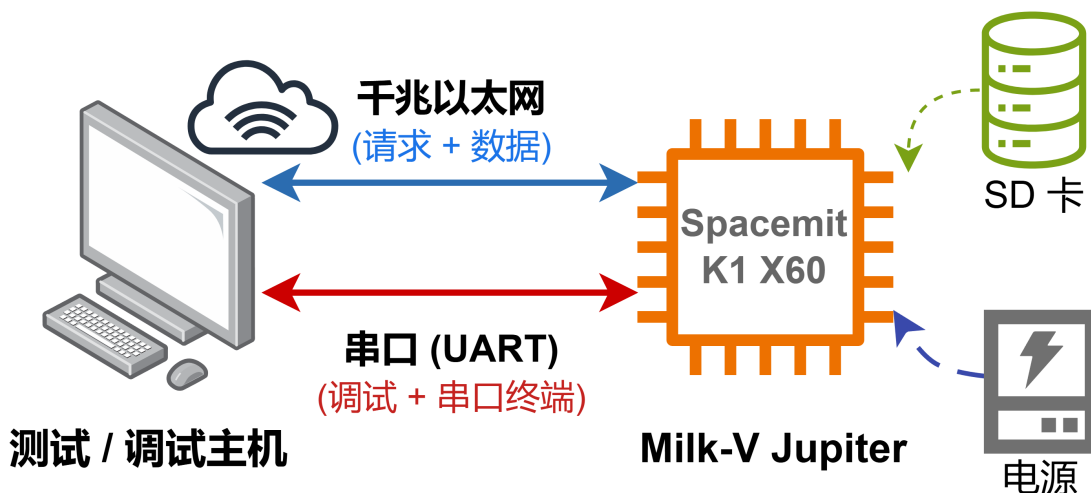


图 4.1 测试环境硬件连接示意图

完成上述测试环境所需的主要硬件如下：

- Milk-V Jupiter (Spacemit X60, 8 核 RVV 1.0) 开发板 1 块
- USB 转串口模块 1 个
- DC 电源适配器 (5.5 × 2.5 mm, 12 V / 3 A 以上) 1 个
- 以太网双绞线若干
- 杜邦线若干
- SD 卡 1 张
- 读卡器 1 个

Milk-V Jupiter 的供电、网络 and 串口连接实物如图 4.2 所示。

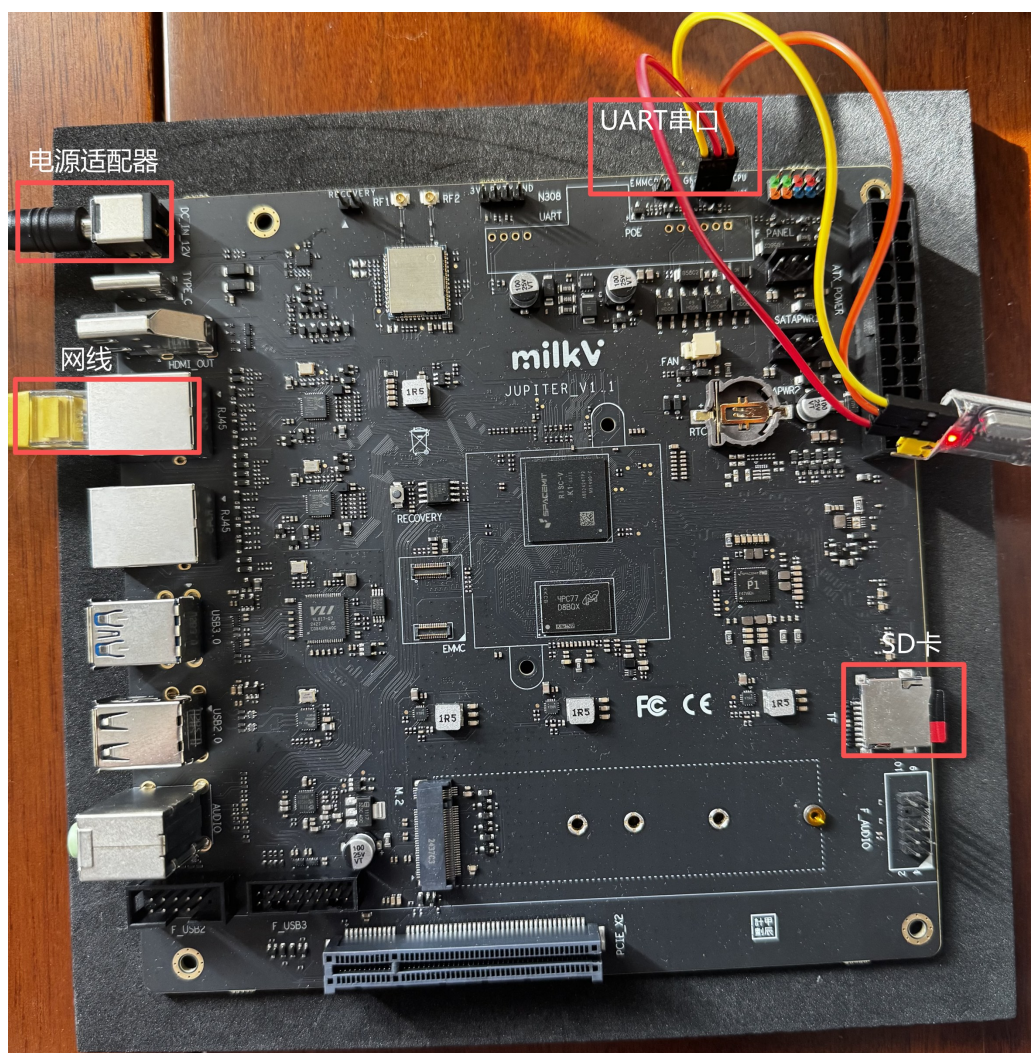


图 4.2 Milk-V Jupiter 开发板实物连接

作为推理负载与内核观测的实际承载平台，Milk-V Jupiter 的核心技术参数见表 4.1。

表 4.1 Milk-V Jupiter 核心技术参数

项目	规格
SoC	Spacemit X60 (K1), 8 核 RISC-V
RVV 支持	RVV 1.0, VLEN = 256
主频	1.6 GHz
内存	8 GB LPDDR4X
存储	MicroSD 卡 (系统) 与 M.2 NVMe SSD (模型及结果)

4.1.2 软件环境

测试主机采用 Windows 与 WSL 组合环境运行 Python 评测程序，主要承担远程控制、HTTP 请求生成和数据处理，其软件环境见表 4.2。

表 4.2 测试主机软件环境

组件	版本/配置
操作系统	Windows 11/10、WSL
运行环境	Python 3.x
依赖库	requests、numpy、paramiko

推理服务端部署启用 RVV 支持的 llama.cpp，并使用相同量化格式的 1B 和 3B 指令微调模型构造不同计算规模的推理负载。六种通用调度方法正式性能评测所采用的软件环境见表 4.3。

表 4.3 推理服务端软件环境

组件	版本/配置
操作系统	openEuler/BianbuOS, 内核 6.6
推理框架	llama.cpp (GGML_RVV=ON, GCC 14 交叉编译)
模型 1	Llama 3.2 1B Instruct Q4_K_M (GGUF)
模型 2	Llama 3.2 3B Instruct Q4_K_M (GGUF)

需要说明的是，六种通用调度方法的正式对比在 Linux 6.6 环境中完成；sched_ext 自定义调度器则运行于重新编译并启用 CONFIG_SCHED_CLASS_EXT 等配置的 Linux 6.18.38 内核。后者主要用于验证自定义选核、队列派发、时间片分配及安全回退机制。由于两类试验所使用的内核版本和参数配置不同，相关数据在后文分别呈现，不进行脱离实验条件的直接数值比较。

4.2 测试说明

本项目的测试工作从功能正确性验证和性能量化评测两个层面展开，分别对应模块测试和集成测试。前者用于确认各组成模块具备进入完整试验流程的基本条件，后者用于观察不同调度配置下系统级指标的变化。

模块测试分别验证各组件的基本功能，确认其能够独立运行并产生可供复核的输出。验证内容包括推理服务能否正常启动和响应请求、RVV 向量化编译是否生效，bpfftrace 能否观测满足 V 状态保存或恢复条件的代理事件并输出统计结果，量化工具能否生成字段完整的性能数据，以及各项调度策略能否正确配置并作用于目标进程。

模块测试不评价不同方案的性能优劣，也不涉及模块之间的完整联动，其目的在于进入集成测试前排除服务启动、观测链路、数据输出和策略部署等基础故障。

集成测试将推理服务、调度配置、请求生成、V 状态追踪和数据归档串联为完整的端到端流程。六种通用调度方法以 taskset 作为受控参考组，在相同参数矩阵下分别采集吞吐量（TPS）、首 token 延迟（TTFT）、端到端延迟（E2E）和 V 状态条件事件等指标，以比较核心约束、资源隔离及调度策略带来的变化。per-CPU 统计用于观察事件在各处理器核心上的分布，并辅助检查核心约束是否按预期生效。全部通用方案均由 bench_runner.py 自动化框架统一执行，以保证测试顺序、请求负载、数据采集和清理过程保持一致。sched_ext 原型则采用独立配置验证其加载、运行、卸载及不同负载下的表现，其结果不并入六种通用方法的统一排名。

试验选用 Instruct 模型而非 Base 模型，主要原因在于指令微调模型能够较稳定地遵循输入要求，并生成结构相对一致的连续文本。Base 模型的训练目标以语言建模为主，面对指令式提示时更容易表现为开放式续写，使生成长度和终止行为产生额外波动，不利于构造可重复的推理负载。采用 Instruct 模型有助于使各轮请求遵循相近的生成过程，从而减少任务语义差异对 TPS、TTFT 和 E2E 等指标的干扰，提高试验结果的可比性。

实验平台基于 SpacemiT K1 芯片，并按照项目预设将 CPU 0-3 划分为 AI 计算核心域，将 CPU 4-7 划分为通用核心域。六种 CPU 亲和性、资源隔离和调度策略方案统一限制在 CPU 4-7 上运行，用于在相同核心范围内比较不同 Linux 机制对推理线程调度行为的影响。sched_ext 自定义调度器则将推理工作线程引导至 CPU 0-3，并将普通任务分配至 CPU 4-7，用于验证专用线程识别和核心分区策略。由于后者同时改变了内核版本、核心范围和调度逻辑，其结果仅在自身的低负载与高负载配置内进行前后比较，不与六种通用方案直接排名。

4.3 测试步骤

测试过程主要由 x86 测试主机发起，并通过 SSH 和 HTTP 接口控制推理服务端。具体操作分为模块测试和集成测试两部分：模块测试用于逐项核对功能链路，集成测试则在统一参数和自动化流程下完成重复试验。

4.3.1 模块测试

模块测试按照推理服务、追踪脚本、量化工具和调度策略的顺序展开，各项测试相互独立，不以其他模块的完整联动为前提。

llama-server 的验证从模型加载和服务启动开始。首先检查启动日志是否完整，并确认其中包含 `RISCV_V = 1`，以核对当前程序已启用 RVV 编译支持；随后使用 curl 请求 `/health` 端点，检查服务是否返回 HTTP 200 状态码；最后发送单次推理请求，确认 SSE 响应能够正常结束，生成结果和 token 统计字段均完整返回。

bpfttrace 功能验证在开发板上独立运行 `vstate_trace_cpu.bt`，首先检查探针能否成功挂载，并确认 interval 模式能够以 10 秒为周期输出统计快照。脚本运行期间触发少量推理请求，观察 `saves_total` 和 `restores_total` 等累计计数是否更新，同时检查进程级与 per-CPU 统计项能否正常生成。该步骤用于验证观测链路和输出格式，不以计数增长本身证明代理事件与硬件保存动作完全等价。

量化工具的验证在推理服务和 `/health` 端点均可访问的条件下进行。测试选取单档并发配置和少量重复次数，依次检查 HTTP 请求发送、SSE 响应解析、异常状态记录及 CSV 文件写入过程。输出文件应包含吞吐量、TTFT、E2E、token 数量和试验标识等字段，且不得出现无故缺失、负值或时间顺序异常。

通用调度策略首先以独立进程作为测试对象，分别配置 CPU 亲和性、cgroup 核心范围、调度分区和实时调度属性。配置完成后，使用 `taskset -p`、`/proc/<pid>/status` 中的 `Cpus_allowed_list`、相关 cgroup 控制文件及 `chrt -p` 交叉核对允许 CPU 列表和调度优先级，并结合线程运行位置采样确认约束已作用于目标进程。

自定义调度器通过 `./vec_affine_sched -t libggml-cpu.so` 加载。加载前后分别执行 `bpftool struct_ops show`，检查 `vec_affine_ops` 的注册状态，并读取 `/sys/kernel/sched_ext/state`，确认调度器运行期间的状态为 `enabled`。随后启动推理负载，观察 Loader 中选核、入队和派发计数是否更新；最后向 Loader 发送终止信号，再次核对注册项和 `sched_ext` 状态，以验证卸载与默认调度恢复流程。

4.3.2 集成测试

集成测试以 `bench_runner.py` 作为统一执行入口，将 `llama-server`、`bpfttrace`、调度配置与量化工具串联为完整流程。为保证不同方法之间具有可比性，每批试验开始前需固定以下三类配置：

- 评测工具配置：固定 SSH 连接参数，包括开发板 IP、用户名和认证信息，并统一模型文件、`bpfttrace` 脚本及 `llama.cpp` 程序的路径。
- 推理负载配置：设置总请求数、并发度、`prompt` 长度（64/128/256 三档）、输出长度、上下文长度、推理线程数和并行槽位数。
- 调度环境配置：依据待测方法设置 CPU 亲和性、`cgroup`、`systemd` 或调度策略参数，并在试验开始前检查上一轮配置是否已清除。

完成上述配置后，每轮试验按照以下顺序执行：

1. 按照当前参数和调度方法构造启动命令，在开发板上启动 `llama-server`，并自动轮询 `/health` 端点直至服务就绪。
2. 启动 `bpfttrace` 后台采集，确认探针成功挂载，并记录请求发送前的累计计数基线。
3. 按当前并发度向服务端发送总计 50 个 HTTP 推理请求，并采用流式 SSE 协议接收响应。
4. 等待全部请求完成，记录每个请求的发送时间、首 token 到达时间、完成时间及生成 token 数。
5. 停止 `bpfttrace` 并读取最终累计计数，以终值减去基线，得到本轮 V 状态保存与恢复条件事件的净增量。
6. 关闭 `llama-server`，检查并终止残留进程，同时清理 `cgroup` 目录或 `systemd` 运行时配置。
7. 将请求级明细及本轮汇总结果写入 CSV 文件，汇总字段包括 TPS、平均 TTFT、P95 TTFT、E2E 和 V 状态条件事件。
8. 确认环境恢复后切换至下一种调度方法，并重复上述流程；每种方法在相同参数配置下独立执行 3 次。

全部试验完成后，按参数配置和调度方法汇总结果，计算重复轮次的均值与标准差；P95 TTFT 则由同组请求样本统一汇集后计算。`per-CPU V` 状态条件事件用于辅助检查核心约束：若目标进程在非预期核心上出现相关记录，则标记为 LEAK 候选，并结合允许 CPU 掩码和

线程运行位置进一步复核，而不直接将单一计数判定为隔离失效。最终生成对比表格和图形，综合分析各策略下的 TPS、平均 TTFT、P95 TTFT 及归一化 V 状态指标。

4.4 测试结果

本节首先给出推理服务、追踪工具、量化工具和调度策略的功能验证结果，随后分别分析六种通用调度方法及 sched_ext 自定义调度器的集成测试数据。

4.4.1 模块测试

模块测试覆盖推理请求响应、V 状态条件事件观测、性能数据生成和调度配置生效四个环节，其结果用于确认各模块具备进入集成测试的基本条件。

4.4.1.1 llama.cpp 功能验证

llama.cpp 以 server 模式在 Milk-V Jupiter 上正常启动，Llama 3.2 1B 和 3B 两种量化模型均能完成加载。启动日志中包含 RISC_V = 1 标志，表明当前构建已启用 RVV 支持。使用 curl 向 /health 端点发送 HTTP GET 请求后，服务返回 200 状态码；进一步发送单次推理请求，能够获得完整的流式生成结果和 token 统计信息。相关日志如图 4.3 所示。

```

root@milkv-jupiter:~# ./llama-server-rvv -m Llama-3.2-3B-Instruct-Q4_K_M.gguf --port 8080 -c 8192 --host 0.0.0.0 --parallel 8 --lv 4
0.00.020.892 I cmn common_param: common_params_print_info: build 0 (unknown) with GNU 14.2.0 for Linux riscv64
0.00.020.925 I cmn common_param: device_info:
0.00.020.927 I cmn common_param: CPU : SpaceTime(R) X60 (7834 MiB, 7834 MiB free)
0.00.021.200 I cmn common_param: system_info: n_threads = 8 (n_threads_batch = 8) / 8 | CPU : RISC_V = 1 | RVV_VLEN = 32 | LLAMAFILE = 1 | OPENMP = 1 | REPACK = 1 |
0.00.021.609 I srv init: using 12 threads for HTTP server
0.00.022.397 I srv start: binding port with default address family
0.00.024.124 I srv load_model: loading model 'Llama-3.2-3B-Instruct-Q4_K_M.gguf' 模型名
Cutting Knowledge Date: December 2023
Today Date: 07 Jul 2026

You are a helpful assistant<|eot_id|><|start_header_id|>user<|end_header_id|>
Hello<|eot_id|><|start_header_id|>assistant<|end_header_id|>
Hi there<|eot_id|><|start_header_id|>user<|end_header_id|>
How are you?<|eot_id|><|start_header_id|>assistant<|end_header_id|>
.
0.09.192.696 I srv init: init: chat template, thinking = 0
0.09.192.789 I srv llama_server: model loaded
0.09.192.800 I srv llama_server: listening on http://0.0.0.0:8080
0.09.192.813 I srv update_slots: all slots are idle Server成功启动

# administrator @ Morayaki in /mnt/f/Project/OSComp [17:05:36]
$ curl http://192.168.1.103:8080/health
{"status": "ok"} 网络请求成功

```

图 4.3 llama-server 启动日志与 curl 请求结果

4.4.1.2 bpfttrace 功能验证

vstate_trace_cpu.bt 脚本能够在开发板上正常运行，相关探针成功挂载至调度切换事件，并在 interval 模式下每 10 秒输出一次统计快照。脚本运行期间触发推理请求后，saves_total 和 restores_total 等计数随执行过程更新，进程级和 per-CPU 统计项也能够正常输出，说明追踪与数据汇总链路已经贯通。需要强调的是，这些计数表示满足 V 状

态保存或恢复条件的代理事件，不等同于对底层寄存器保存函数执行次数的直接测量。运行日志如图 4.4 所示。

```

root@milkv-jupiter:~# bpfttrace /root/vstate_trace.bt
Attaching 3 probes...
--- 17:12:45 ---
saves total: @saves_total: 197
restores total: @restores_total: 196
saves by proc:
@saves[llama-server-rv, 2386]: 15
@saves[llama-server-rv, 2443]: 16
@saves[llama-server-rv, 2447]: 16
@saves[llama-server-rv, 2445]: 23
@saves[llama-server-rv, 2444]: 25
@saves[llama-server-rv, 2448]: 30
@saves[llama-server-rv, 2446]: 32
@saves[llama-server-rv, 2449]: 41
restores by proc:
@restores[llama-server-rv, 2386]: 15
@restores[llama-server-rv, 2443]: 16
@restores[llama-server-rv, 2447]: 16
@restores[llama-server-rv, 2445]: 23
@restores[llama-server-rv, 2444]: 25
@restores[llama-server-rv, 2448]: 30
@restores[llama-server-rv, 2446]: 33
@restores[llama-server-rv, 2449]: 40
===== SUMMARY =====
saves total: @saves_total: 254
restores total: @restores_total: 247
saves by proc:
@saves[llama-server-rv, 2447]: 17
@saves[llama-server-rv, 2443]: 18
@saves[llama-server-rv, 2444]: 26
@saves[llama-server-rv, 2445]: 29
@saves[llama-server-rv, 2386]: 30
@saves[llama-server-rv, 2446]: 39
@saves[llama-server-rv, 2448]: 40
@saves[llama-server-rv, 2449]: 55
restores by proc:
@restores[llama-server-rv, 2447]: 16
@restores[llama-server-rv, 2443]: 17
@restores[llama-server-rv, 2444]: 25
@restores[llama-server-rv, 2445]: 28
@restores[llama-server-rv, 2386]: 29
@restores[llama-server-rv, 2446]: 39
@restores[llama-server-rv, 2448]: 39
@restores[llama-server-rv, 2449]: 54

```

图 4.4 bpfttrace V 状态追踪运行日志

4.4.1.3 量化工具功能验证

benchmark_llama.py 在指定并发配置下能够完成请求发送、SSE 响应解析和结果汇总，生成的 CSV 文件包含吞吐量、TTFT、E2E、token 数量和 V 状态条件事件等字段，未出现请求记录缺失或时间值异常。相应的量化结果见第二章表 2.2 和表 2.3。

4.4.1.4 调度策略功能验证

调度策略功能验证分为通用策略配置检查和 sched_ext 原型生命周期检查两部分。对于 CPU 亲和性、实时调度优先级和隔离范围，分别使用 taskset -p、chrt -p 以及 /proc/<pid>/status 交叉核对允许 CPU 列表与调度属性，并通过线程级运行位置采样观察配置后的实际执行范围。推理服务线程的采样结果如图 4.5 所示。

```

root@milkv-jupiter:~# ps -T -p $(pidof llama-server) -o spid,comm,psr,pcpu
  SPID  COMMAND  PSR  %CPU
3385377 llama-server 6  98.9
3385380 llama-server 5   0.0
3385381 llama-server 4   0.0
3385382 llama-server 7   0.0
3385383 llama-server 7   0.0
3385384 llama-server 5   0.0
3385385 llama-server 7   0.0
3385386 llama-server 4   0.0
3385387 llama-server 6   0.0
3385388 llama-server 4   0.0
3385389 llama-server 5   0.0
3385567 llama-server 7   0.0
3385569 llama-server 7   0.0
3385572 llama-server 7   0.0
3385574 llama-server 7   0.0
3385577 llama-server 7   0.0
3385578 llama-server 7   0.0
3385580 llama-server 7   0.0
3385583 llama-server 7   0.0
3385586 llama-server 7   0.0
3385587 llama-server 5   0.0
3385590 llama-server 7   0.0
3385591 llama-server 7   0.0
3385594 llama-server 4   0.0
3385595 llama-server 4   0.0
3385596 llama-server 5   0.0
3385597 llama-server 7   0.0
3417976 llama-server 4  100
3417977 llama-server 5  100
3417978 llama-server 7  100
root@milkv-jupiter:~#

```

图 4.5 推理服务线程在 CPU 4-7 上的运行位置快照

图 4.5 中的 PSR 字段记录了各线程最近一次运行所在的处理器。在该采样时刻，截图中列出的 llama-server 线程均运行于 CPU 4-7，处于高负载状态的计算线程也未超出这一核心范围。由于 PSR 反映的是瞬时运行位置，而不是允许 CPU 掩码，因此该结果需与 taskset 和 /proc/<pid>/status 给出的允许 CPU 列表结合使用，用于确认静态核心约束已经落实到线程执行层面；单次采样本身不用于推断整个运行周期内的迁移行为或性能收益。

对于自定义调度器，验证重点由静态配置转向注册、回调执行和卸载三个环节。加载前后分别执行 bpftool struct_ops show，用于核对 vec_affine_ops 的注册状态；运行期间观察 Loader 输出的累计计数，以确认调度回调已经进入执行路径；发送终止信号后再次查询注册项，用于检查卸载是否完成。如图 4.6 所示，调度器加载后能够查询到对应的 struct_ops，累计入队计数随运行过程递增，终止后注册项消失，由此形成加载、运行和卸载的基本功能闭环。由于截图中的推理线程识别和 AI 核心选取计数仍为 0，该证据仅用于确认 sched_ext 生命周期及基础回调链路，不用于证明推理线程放置效果或性能收益。

```

root@openEuler-riscv64 /home/gh/workspace/mysched/vec_affine3 # bpftool struct_ops show
root@openEuler-riscv64 /home/gh/workspace/mysched/vec_affine3 # ./vec_affine_sched -t /home/gh/workspace/
llama/debug/install_output/lib/libggml-cpu.so &
[1] 92872
root@openEuler-riscv64 /home/gh/workspace/mysched/vec_affine3 # 调度器已成功加载并开启!
成功: uprobe 已精准挂载至 /home/gh/workspace/llama/debug/install_output/lib/libggml-cpu.so 的 ggml_compu
e_forward_mul_mat!
[01:27:28] enqueue_total=3962 select_worker_ai_core=0 select_other_core=34 inference_threads=0 protect_wi
root@openEuler-riscv64 /home/gh/workspace/mysched/vec_affine3 # bpftool struct_ops show
435: vec_affine_ops sched_ext_ops
root@openEuler-riscv64 /home/gh/workspace/mysched/vec_affine3 # [01:27:33] enqueue_total=7693 select_work
er_ai_core=0 select_other_core=85 inference_threads=0 protect_window=0
fg
./vec_affine_sched -t /home/gh/workspace/llama/debug/install_output/lib/libggml-cpu.so
^C[01:27:37] enqueue_total=11688 select_worker_ai_core=0 select_other_core=109 inference_threads=0 protec
t_window=0
正在卸载调度器, 回退至系统原生 CFS 调度器...
root@openEuler-riscv64 /home/gh/workspace/mysched/vec_affine3 # bpftool struct_ops show
root@openEuler-riscv64 /home/gh/workspace/mysched/vec_affine3 #

```

图 4.6 自定义调度器加载、运行与卸载验证

4.4.2 集成测试

集成测试覆盖 taskset（受控参考组）、cpuset、partition-root、systemd-scope、taskset-chrt-b（SCHED_BATCH）和 taskset-chrt-f（SCHED_FIFO）六种方法。每种方法在 6 组参数配置下独立执行 3 次，共形成 108 轮试验和 5400 条请求记录。

数据检查发现，每轮试验开始阶段通常有 2–4 个请求呈现明显高于稳态水平的 TTFT，约占总请求数的 4%。这些样本的 TTFT 为稳态值的 15–61 倍，其出现时机与模型加载后的首次 prompt evaluation、内存分配、缓存填充及 KV Cache 初始化等一次性活动相一致，因此稳态指标分析中单独处理。

以 taskset 在一组数据中的表现为例，冷启动对延迟指标的影响见表 4.4。

表 4.4 冷启动延迟影响示例

指标	含冷启动	排除冷启动	偏差
Avg TTFT	4354 ms	2127 ms	+105%
P95 TTFT	2162 ms	2153 ms	基本不变

排除冷启动样本后，六种方法的稳态平均 TTFT 数值较为接近，具体结果如表 4.5 所示。

表 4.5 性能概览

方法	TPS	Avg TTFT (ms)	P95 TTFT (ms)	V-Saves	V/ktok
taskset（基准）	0.90	3146 ± 335	4802	5663980	131111
cpuset	0.90	3135 ± 343	4787	5566101	128845 (-1.7%)
partition-root	0.90	3120 ± 333	4777	2288749	52980 (-59.6%)
systemd-scope	0.90	3125 ± 351	4723	9072549	210013 (+60.2%)
taskset-chrt-b	0.90	3128 ± 253	4786	5062738	117193 (-10.6%)
taskset-chrt-f	0.86	3279 ± 283	4967	503062	11645 (-91.1%)

自定义调度器在低负载和高负载配置下的独立验证结果如表 4.6 所示。该组试验的内核环境和参数配置与六种通用方法不同，因而仅在相同负载类型内部比较启用调度器前后的变化。

表 4.6 自定义调度器在不同负载下的性能结果

负载类型	调度策略	Avg TTFT (ms)	P95 TTFT (ms)	TPS
低负载	原始	574 ± 109	704	2 ± 0.02
	自定义调度器	533 ± 99	684	2.03 ± 0.02
高负载	原始	2260 ± 583	3439	0.5 ± 0.01
	自定义调度器	1730 ± 1649	4944	1.02 ± 0.22

4.4.2.1 性能分析

从吞吐表现看，六种方法在各组参数配置内的 TPS 波动系数为 1.8%–2.4%。taskset-chrt-f 的 V/ktok 较受控参考组降低 91.1%，其 TPS 同时下降约 4% (0.90→0.86)。在本次 llama.cpp RVV 量化推理环境中，六种方法的吞吐差异总体有限，表明调度策略对吞吐的影响小于模型计算和访存等主要执行开销。

从 V 状态条件事件看，当 TPS 和稳态 TTFT 数值接近时，V-Saves 及其归一化指标 V/ktok 能够从调度行为角度进一步区分各方法。taskset-chrt-f 的 V/ktok 为 11645，较参考值 131111 降低 91.1%，并在 6 组配置中保持最低；partition-root 的 V/ktok 为 52980，较参考值降低 59.6%，在不使用实时优先级的方案中降幅较大。taskset、cpuset 和 taskset-chrt-b 的 V/ktok 位于 117193–131111 之间，变化幅度不超过 11%；systemd-scope 则升至 210013，较参考值增加 60.2%。在当前实验条件下，独立调度分区和实时调度与较少的 V 状态条件事件相关，而单纯的核心绑定或批处理调度所对应的变化相对有限。

对于自定义调度器，低负载条件下启用前后的平均 TTFT 均处于 530–570 ms 区间，TPS 也较为接近。这表明在当前低负载配置中，自定义选核和时间片策略尚未带来明显的性能差异。

在高负载配置下，启用自定义调度器后 TPS 由 0.50 提高至 1.02，增幅约为 104%；平均 TTFT 由 2260 ms 降至 1730 ms，降幅约为 23.5%。与此同时，P95 TTFT 由 3439 ms 上升至 4944 ms，平均 TTFT 的离散程度也明显增大。该结果显示自定义调度器在此配置下提高了吞吐，但同时伴随更强的尾延迟波动，因而不能仅依据平均值判断其综合收益。

从调度机制看，一种可能的解释是，高负载下推理工作线程需要较长的连续运行时间才能完成计算批次。30 ms 时间片和 AI 核心域内的原核心优先策略可能减少计算过程中的调度轮转与跨核迁移，使线程在单次运行期间完成更多张量计算任务，从而提高整体吞吐。另

一方面，较长时间片和队列首部入队策略也可能延长部分请求的等待时间，这与 P95 TTFT 上升的现象一致。由于当前数据尚未分别量化上述因素的独立贡献，该机制解释仍需结合更多重复试验和调度轨迹进一步验证。

4.4.2.2 方法对比总结

六种调度方法的 V 状态条件事件与 TTFT 对比见表 4.7。

表 4.7 方法对比总结

排名	方法	V/ktok (次)	V 较基准	TTFT (ms)	TTFT 较基准
1	taskset-chrt-f	11645	-91.1%	3279	+4.2%
2	partition-root	52980	-59.6%	3120	-0.8%
3	taskset-chrt-b	117193	-10.6%	3128	-0.6%
4	cpuset	128845	-1.7%	3135	-0.3%
—	taskset (基准)	131111	0%	3146	0%
6	systemd-scope	210013	+60.2%	3125	-0.7%

六种调度方法在 TPS 和稳态 TTFT 上的数值差异较小，主要区别体现在 V 状态条件事件上。按照 V/ktok 指标，taskset-chrt-f (SCHED_FIFO) 较参考组降低 91.1%，为本次试验中降幅最大的方案；partition-root (cgroup v2 独立调度分区) 降低 59.6%，在不采用实时优先级的方案中降幅最大。systemd-scope 的 V/ktok 较参考组增加 60.2%，在当前配置下未表现出抑制 V 状态条件事件的优势。

表中名次仅按照归一化指标 V/ktok 从低到高排列，taskset 作为受控参考组不参与名次竞争。该指标将 V 状态保存条件事件数按生成 token 数归一化，以减弱请求规模差异的影响；TTFT 较参考组一列则用于同步观察事件数量变化是否伴随可见的首 token 延迟变化。因此，该排序仅反映本次 Milk-V Jupiter、Linux 6.6、指定模型和并发配置下的 V 状态条件事件抑制效果，不代表跨平台、跨模型的综合性能排名。

需要说明的是，表 4.7 汇总的是六种通用调度方法在同一参数矩阵下的结果；表 4.6 中的 sched_ext 自定义调度器采用独立的低负载和高负载配置，内核环境与实验口径均不相同，因而未纳入同一排名。前者用于比较现有 Linux 机制对 V 状态条件事件的影响，后者用于观察专用调度原型在不同压力水平下的运行表现，两组数值不作直接横向排序。

第 5 章 总结与展望

5.1 总结

围绕赛题提出的三项核心要求及相关可选任务，本项目在 Milk-V Jupiter（Spacemit K1，RVV 1.0）平台上完成了量化分析工具、调度优化方案和自动化评测流程的设计、实现与验证。赛题要求与项目实现成果的对应关系如表 5.1 所示。

表 5.1 赛题要求与评分项完成情况

赛题要求与评分要点	项目完成情况
赛题基本要求	
基于至少一种国内主流开源操作系统开展开发、编译、运行与测试。	✓ 已完成。在 Milk-V Jupiter 平台的 BianbuOS 与 openEuler 环境中完成 RVV 推理服务部署、V 状态观测和性能测试。
问题量化：建立可复现的 V 状态保存/恢复路径观测方法，分析其与并发度、模型规模及推理性能之间的关系。	✓ 已完成。基于 bpftrace 实现了 V 状态条件事件观测，能够统计保存/恢复条件事件、完成线程与 CPU 归因，并对相关调度切换路径进行计时；在 1B、3B 模型和 2、4、8 并发配置下完成了基线采集与量化分析。
调度优化：实现至少一种 CPU affinity、cpuset、sched_ext 或同等功能的操作系统调度优化机制。	✓ 已完成。实现并验证了六种通用调度策略，同时完成了基于 sched_ext BPF 的自定义调度原型。
性能评测：在真实 RVV 1.0 硬件上，采用参数规模不超过 3B 的量化模型，在 2、4、8 并发配置下开展重复测试，并对比 TPS、平均 TTFT 和 P95 TTFT。	✓ 已完成。在 Milk-V Jupiter 真实 RVV 1.0 硬件上，采用 Llama 3.2 1B、3B Q4_K_M 模型完成了多轮重复试验，并统一归档 TPS、平均 TTFT、P95 TTFT、V 状态条件事件和单请求明细 CSV。
可选完成项	
与 x86 AVX2/AVX-512 平台进行横向对比，分析 RVV VLA 设计在 V 状态切换问题上的特殊性。	✓ 已完成。在 AMD Ryzen 7 8745H 的 AVX-512 环境中完成了 3B 模型、4 并发配置下的观测，并对比分析了 RVV 的 VS 状态机制与 x86 的 XSAVE/XSTATE_BV 状态管理及保存路径。
项目代码评分项	

续表

量化分析工具功能：统计 V 状态保存/恢复条件事件、相关调度路径事件或 Dirty V 线程被抢占事件，并对其开销进行量化。	✓ 已完成。实现了 V 状态追踪脚本与自动化采集流程，在 sched_switch 处识别满足 Dirty 条件的任务切换，并通过 finish_task_switch 路径计时估算涉及 V 状态任务的切换开销。
调度优化策略功能：至少一种调度策略功能完整且能够正常运行。	✓ 已完成。六种通用调度策略与 sched_ext 自定义调度原型均已纳入自动化评测流程，并完成了相应的功能验证。
代码正确性与性能：在多种并发场景下验证优化效果的稳定性与可重复性。	✓ 已完成。已使用不同规模的模型，在 2、4、8 并发配置下开展重复测试，并通过统一的参数配置与结果归档流程保证试验过程可复现。
代码风格、注释、质量与构建说明。	✓ 已完成。提供了包含必要注释和 Markdown 说明文档的自动化评测脚本、trace 追踪脚本、自定义调度配置及通用策略配置，可用于复现主要测试流程。
技术报告评分项	
设计方案：准确分析 V 状态切换问题，并阐明其与 Linux 调度机制之间的关系。	✓ 已完成。分析了 RVV Dirty 状态、上下文切换路径、CFS 调度竞争与线程迁移之间的关系，并说明了 SCHED_BATCH、SCHED_FIFO 和 sched_ext 的适用场景。
实现方案：完整说明系统架构、关键流程、实现细节及相关技术调研。	✓ 已完成。给出了用户层、内核层和 CPU 层的系统架构，说明了 llama-server 请求处理、V 状态追踪、调度策略部署及结果归档流程，并补充了 sched_ext、RVV 与 x86 状态管理机制的相关调研。
运行效果与测试结果：规范说明测试环境、统一基线、重复试验、结果归档及可复现流程。	✓ 已完成。完整记录了软硬件环境、模型与服务参数、并发配置、CPU 调度策略和原始试验结果，并提供了相应的实验演示视频。
特色创新：提出新的调度原语或用户态-内核态协同机制，或形成更深入的问题分析。	✓ 已完成。在常规核心绑定和调度类参数调整之外，实现了基于 sched_ext BPF 的自定义调度原型。

续表

代码提交情况：Git 提交历史规范、工作量分布合理，提交信息能够清晰反映项目迭代过程。	✓ 已完成。 
团队协作情况：成员分工明确，协作机制有效。	✓ 已完成。团队通过飞书开展任务分工、技术讨论与进度同步，并使用 Git 进行版本管理，通过提交记录持续跟踪各模块的开发、集成与测试进展。 工作： xsave/xrstor 会冲刷 L1/L2 cache line，破坏矩阵乘法（GEMM）的局部性 kv unified https://github.com/ggml-org/llama.cpp/discussions/21961 1. sched_ext调研，bpf用户端，策略。郭恒 @sched_ext使用手册 a. 策略： i. prev_cpu亲和+超时时间 ii. llama标记处理完成，完成后，调度。 iii. Decode和Prefill，其他线程隔离开，Decode和Prefill只在某一个核心，PD任务在没有slot可用，-poll 自选等待，而不是换入换出，所decode，prefill，D池(固定CPU 0 1)，P池，其他池。自旋等待 iv. 主要rootfs移植，sched-ext，包，系统出现问题【】 2. llama.cpp PD分离调研 陈子轩 a. --continus batching找出分离PD的地方（kv unified） 3. -Cr --cpu-strict 调研 汪坤 a. 改完了，llama server 郭恒 (RVV) (v) b. Group 6 两组 -Cr -Cr --cpu-strict 郭恒 (v) 4. x86向量运算V状态对比量化 汪坤 a. x86位图保存，存储逻辑，给出解释 (v) b. x86测试结果贴上去 (v) c. riscv，restore恢复时机不同，看能否追踪到。（？）无法 d. 测试dirty时间（？）cpu cycle 在什么地方结束？首先binbu内核没开kprobe，需要重编内核开启，之前脚本使用的只是tracepoint，其次vsave操作之后最近能插桩的点从vsave之间也会有噪声 24开会

在当前 RVV 1.0 硬件（Spacemit K1，VLEN=256）和 llama.cpp 量化推理场景下，六种通用调度方法的 TPS 与稳态 TTFT 数值差异总体较小，方法间的区别主要体现在 V 状态条件事件代理指标上。各方法的 V/ktok 分布于 11645–210013 之间；相较于受控参考组，taskset-chrt-f（SCHED_FIFO）的 V/ktok 降低 91.1%，在本次试验中降幅最大，但其 TPS 同时由 0.90 降至 0.86；partition-root（cgroup v2 独立调度分区）的 V/ktok 降低 59.6%，在不采用实时优先级的方案中降幅最大。上述结果表明，调度策略能够显著改变 V 状态条件事件的发生频度，但该指标的下降并不必然转化为吞吐率或首 token 延迟的同步改善。

5.2 未来工作

后续研究首先需要统一实验环境与证据口径。当前六种通用调度方法的正式评测基于 Linux 6.6，而 `sched_ext` 原型在独立的新内核环境中完成验证，两组结果尚不能直接横向比较。未来将把基线方法与自定义调度器迁移至同一内核、根文件系统和推理参数配置，在保持模型、并发度及 CPU 频率等控制变量一致的条件下重新开展重复试验，从而减少内核版本和系统配置差异对结论的干扰。

在观测机制方面，现有 `bpfttrace` 工具主要统计满足 V 状态保存或恢复条件的代理事件，尚不能完整分离向量寄存器保存、任务迁移、缓存失效和队列等待等因素的独立影响。后续可结合内核 `tracepoint`、`eBPF` 调度轨迹与硬件性能计数器，建立从请求、线程到 CPU 的端到端关联，并通过增加重复次数、报告置信区间和设置针对性的调度干预试验，进一步分析 V 状态条件事件与吞吐率、TTFT 及尾延迟之间的因果关系。

在调度策略方面，将基于 `sched_ext` 的动态装载、调度队列和异常回退能力，进一步完善 V 状态感知的自定义调度器。现有原型采用固定的 30ms 时间片和静态核心分区，后续可综合 Dirty V 状态、原运行核心、请求队列长度以及 `prefill`、`decode` 阶段特征，在线调整时间片、任务优先级和核心分配策略。优化目标也将由单一吞吐率扩展为面向服务等级目标的多目标控制，在提高系统吞吐的同时约束 P95 TTFT、TPOT 和请求饥饿风险，以适应交互式推理对尾延迟稳定性的要求。

在系统验证与成果凝练方面，将逐步扩展至不同核心规模、VLEN、模型规模、量化精度和上下文长度的 RISC-V 平台，并与 x86 AVX-512、ARM SVE 等可变长度或宽向量体系进行对照，以识别 V 状态感知调度的适用边界。在此基础上，公开可复现的测试脚本、调度轨迹和匿名化结果数据，围绕“RISC-V 向量状态开销的系统化量化”与“面向尾延迟约束的 LLM 推理调度”凝练研究问题，形成可供操作系统与体系结构领域论文验证的实验方法和原型系统。

参考文献

- [1] RISC-V International. RISC-V “V” Vector Extension, Version 1.0[EB/OL]. <https://github.com/riscv/riscv-v-spec>.
- [2] ggml-org. 2026a. llama.cpp: build-riscv64-spacemit 编译指南 [EB/OL]. <https://github.com/ggml-org/llama.cpp/blob/master/docs/build-riscv64-spacemit.md> [2026-05-25].
- [3] ggml-org. 2026b. Batch Processing Pipeline[EB/OL]. <https://deepwiki.com/ggml-org/llama.cpp/3.5-batch-processing-pipeline> [2026-07-04].
- [4] Linux Kernel Documentation. Vector Extension Support for RISC-V Linux[EB/OL]. <https://docs.kernel.org/arch/riscv/vector.html>.
- [5] Linux Kernel Documentation. CFS Scheduler[EB/OL]. <https://docs.kernel.org/scheduler/sched-design-CFS.html>.
- [6] ggml-org. 2026d. llama.cpp 服务端任务、slot 与批次处理实现: tools/server/server-context.cpp[EB/OL]. <https://github.com/ggml-org/llama.cpp/blob/master/tools/server/server-context.cpp> [2026-07-29].
- [7] ggml-org. 2026e. llama.cpp 上下文解码、ubatch 与线程参数实现: src/llama-context.cpp[EB/OL]. <https://github.com/ggml-org/llama.cpp/blob/master/src/llama-context.cpp> [2026-07-29].
- [8] QuantFactory. 2024a. Llama 3.2 1B Instruct GGUF[EB/OL]. <https://huggingface.co/QuantFactory/Llama-3.2-1B-GGUF> [2024-09-25].
- [9] QuantFactory. 2024b. Llama 3.2 3B Instruct GGUF[EB/OL]. <https://huggingface.co/QuantFactory/Llama-3.2-3B-GGUF> [2024-09-25].
- [10] Linux Kernel Documentation. EEVDF Scheduler[EB/OL]. <https://docs.kernel.org/scheduler/sched-eevdf.html>.
- [11] KERRISK M. 2026. sched(7): Linux manual page[EB/OL]. <https://www.man7.org/linux/man-pages/man7/sched.7.html> [2026-02-08].
- [12] Linux Kernel Documentation. Extensible Scheduler Class[EB/OL]. <https://docs.kernel.org/scheduler/sched-ext.html>.
- [13] RuyiSDK. 2025. Milk-V Jupiter 相关讨论 [EB/OL]. <https://ruyisdk.cn/t/topic/1971> [2025-11].

-
- [14] ISCAS. 2024. Bianbu/Fedora Milk-V Jupiter 镜像仓库 [EB/OL]. <https://openkoji.iscas.ac.cn/pub/dist-repos/dl/Milk-V/Jupiter/images/latest/> [2024-07-15].
- [15] openEuler 社区. 2026. openEuler 25.09 Milk-V Jupiter 镜像 [EB/OL]. http://images.oeuv.ac.cn/release/openeuler/25.09/milkv/jupiter/spacemit-k1-uboot-bootsd?revision=0&lang=zh_CN&doc=0 [2026-01-10].
- [16] Irradium. 2026. irradiumOS Milk-V Jupiter 镜像 [EB/OL]. https://dl.irradium.org/irradium/images/milk_v_jupiter/ [2026-05-19].
- [17] ggml-org. 2025. Issue #12693: RISC-V RVV related[EB/OL]. <https://github.com/ggml-org/llama.cpp/issues/12693> [2025-04-01].
- [18] ggml-org. 2026c. Spacemit IME 环境头文件 [EB/OL]. https://github.com/ggml-org/llama.cpp/blob/master/ggml/src/ggml-cpu/spacemit/ime_env.h [2026-05-14].
- [19] Meta. 2024. Llama 3.1 8B Instruct Q4_K_M GGUF[EB/OL]. https://huggingface.co/tinhvanhung/Meta-Llama-3.1-8B-Instruct-Q4_K_M [2024-12-22].
- [20] TORVALDS L. 2007. kernel/sched_fair.c, Linux v2.6.23[EB/OL]. https://github.com/torvalds/linux/blob/v2.6.23/kernel/sched_fair.c [2007-10-09].
- [21] KERRISK M. 2024. perf(1): Linux manual page[EB/OL]. <https://www.man7.org/linux/man-pages/man1/perf.1.html> [2024-06-20].
- [22] Geek Blogs. 2025. Linux perf 性能分析工具简介 [EB/OL]. <https://geek-blogs.com/blog/linux-perf/> [2025-11-08].
- [23] YIFEI W. 2025. GGML Deep Dive IV: Computation in Context-Only Mode[EB/OL]. <https://medium.com/@yifeiw203/ggml-deep-dive-iv-computation-in-context-only-mode-part-2-cebe30f5ada3> [2025-02-22].
- [24] FunWithLinux. 2026. Setting CPU Affinity of a Process from the Start on Linux[EB/OL]. <https://www.funwithlinux.net/blog/setting-cpu-affinity-of-a-process-from-the-start-on-linux/> [2026-01-04].
- [25] openEuler 社区. 2023. 在 QEMU 上运行 openEuler RISC-V[EB/OL]. <https://www.openeuler.org/zh/blog/phoebe/2023-09-26-Run-openEuler-RISC-V-On-Qemu.html> [2023-10-26].

附录 A RISC-V 与 x86 架构向量寄存器切换的对比

本章从扩展状态的组织方式、保存决策的执行位置以及状态分量的追踪粒度三个方面，对 RISC-V RVV 与 x86-64 的寄存器切换机制进行对比。两种架构均需要在任务切换过程中维护向量计算上下文，但其硬件接口与内核实现路径并不相同：RISC-V 将向量状态是否需要保存的信息直接暴露给软件，而 x86-64 则将各扩展分量的使用状态交由处理器内部维护。明确这一差异，有助于界定两类平台上观测数据的含义，并避免将不同测试环境中的保存次数直接等同为架构开销。

A.1 扩展状态管理机制

x86-64 未采用 RISC-V 标量浮点与向量状态分别管理的方式，而是将 x87、SSE、AVX、AVX-512 等扩展状态统一组织在连续的 XSAVE Area 中。内核通过 XSAVE、XSAVEOPT 或 XSAVES 执行保存，并通过 XRSTOR 或 XRSTORS 完成恢复。与 RVV 向量寄存器地位相近的状态分量由 XSTATE_BV 位图中的 bit 2、bit 5、bit 6 和 bit 7 表示，具体定义见表 A.1。

表 A.1 XSAVE 状态分量 (XSTATE_BV) 位定义

Bit	状态分量	保存内容
0	x87	标量浮点状态（历史兼容分量）
1	SSE	XMM 寄存器（x86-64 强制基线）
2	AVX	YMM0-15 的高 128 位
5	AVX-512 opmask	k0-k7 掩码寄存器
6	AVX-512 ZMM_Hi256	ZMM0-15 的高 256 位
7	AVX-512 Hi16_ZMM	ZMM16-31 的全部 512 位

处理器内部维护的 XINUSE 追踪位用于记录每个状态分量自加载初始配置以来是否被使用。该追踪位不是软件可以直接读写的架构寄存器，操作系统只能在保存指令执行后，通过写入 XSAVE Header 的 XSTATE_BV 间接判断相关分量的使用情况。因此，x86-64 内核在满足调度路径的前置条件后，会调用 `save_fpregs_to_fpstate()`；是否跳过某个具体分量的写回，则由 XSAVEOPT 或 XSAVES 根据处理器内部状态决定。

RISC-V 的处理方式与此不同。内核能够在软件层直接查询 `sstatus.VS`，仅当 VS 为 Dirty 时进入向量状态保存路径。一旦触发保存，当前实现需要写回完整的 32 个向量寄存器。由此可见，RISC-V 的保存判定发生在软件可见、可插桩观测的内核路径中，而 x86-64 对具体分量的跳过决策发生在保存指令内部，两者在观测口径上并不等价。

A.2 观测环境与统计口径

x86-64 侧的补充试验在 WSL2 环境中完成，处理器为 AMD Ryzen 7 8745H，支持 AVX-512。试验使用 `taskset -c 12-15` 将 3B 量化模型的运行范围限制在四个逻辑 CPU 上，并采用四线程、四并发配置。观测程序统计 `save_fpregs_to_fpstate()` 的调用次数，并在函数返回后读取线程 XSAVE Header 中的 `XSTATE_BV`，对 bit 2、bit 5、bit 6 和 bit 7 分别进行归类。

本试验将一次“向量状态保存活动”定义为：`save_fpregs_to_fpstate()` 已被调用，且返回后的 `XSTATE_BV` 在 bit 2、bit 5、bit 6、bit 7 中至少有一位置 1。这里的置位表示相应分量自加载初始配置以来曾被使用，并不必然表示该分量在本次任务切换前刚刚发生修改。该定义与 RISC-V 侧基于 VS Dirty 条件的事件统计含义不同，因此仅用于分析 x86-64 内部各状态分量的活动特征。

试验共观测到 381 次符合条件的保存调用，各分量的置位次数与占比如表 A.2 所示。同一次调用可以同时包含多个置位分量，因此表中各行次数不能相加后解释为总保存次数。

表 A.2 llama-server 扩展状态保存活动统计

XSTATE_BV 位	对应硬件状态	保存次数	占比
bit 2 (0x04)	AVX (YMM0–15, 256 bit)	92	24.1%
bit 5 (0x20)	OPMASK (k0–k7 掩码寄存器)	381	100%
bit 6 (0x40)	ZMM_HI256 (ZMM 高 256 位)	75	19.7%
bit 7 (0x80)	ZMM16–31 (高 16 个 512 bit 寄存器)	381	100%

上述 381 次调用明显少于 RISC-V 板卡在相近并发配置下观测到的条件事件数量，但这一差异不能作为两种架构保存开销的直接比较依据。WSL2 宿主机的处理器性能显著高于 RVV 板卡，相同推理负载的墙钟运行时间更短，相应的线程调度与上下文切换次数也会减少。因而，本节仅比较状态管理机制与分量分布，不依据绝对事件数对两种架构作性能排序。

A.3 观测解释与适用边界

表 A.2 显示，bit 5 (opmask) 与 bit 7 (ZMM16–31) 在每次保存调用后均为 1，而 bit 2 (AVX 高位) 和 bit 6 (ZMM 高 256 位) 仅在部分调用中置位。该分布与“所有向量分量始终同步变化”的直观预期并不一致，说明不同扩展分量的状态清理行为会影响 `XSTATE_BV` 的长期取值。

x86-64 硬件不会在每次任务切换时自动清零全部 `XSTATE_BV` 分量。相关状态通常在 `exec()` 后重新加载初始配置，或由程序执行 `VZEROALL`、`VZERoupper` 等指令时发生变化。编译器生成 AVX 或 AVX-512 代码时，常在向量代码返回到可能仅使用 SSE 的调用路径之前插入 `VZERoupper`，以降低高位状态带来的额外开销；`opmask` 与 `ZMM16-31` 则不属于该指令的清理范围。结合本次观测，可以推测 bit 2 与 bit 6 对应状态更容易被显式清理，而 bit 5 与 bit 7 一旦被使用，便可能持续保持“曾经使用”的置位状态。该解释属于基于架构语义和编译器行为作出的机制推断，后续仍需结合反汇编结果与指令级追踪进一步核验。

从操作系统视角看，RISC-V 的 `sstatus.VS` 是软件可直接读取的单一状态位，内核能够明确观测“是否进入保存流程”，但保存动作以完整向量上下文为单位。x86-64 的 `XINUSE` 由处理器内部维护，内核会进入统一保存路径，具体分量是否被跳过则由指令内部决定，调度层只能通过返回后的 `XSTATE_BV` 间接推断。前者具有清晰的软件观测边界，后者具备更细的硬件分量划分，但这种细粒度能力能否转化为实际收益，还取决于负载是否触发相应分量的清理机制。

两种架构在状态组织、保存判定与可观测性方面的差异可归纳为表 A.3。该对比表强调的是机制边界，而不是跨平台性能结论；其中任一项均需要结合具体内核版本、编译器行为和负载特征解释。

表 A.3 RISC-V RVV 与 x86-64 扩展状态切换机制对比

对比维度	RISC-V RVV	x86-64 AVX/AVX-512
状态标记	软件可读取的 <code>sstatus.VS</code>	处理器内部维护 <code>XINUSE</code> ，并写入 <code>XSTATE_BV</code>
保存判定	内核先判断 <code>VS</code> 是否为 <code>Dirty</code>	内核进入统一保存路径，指令内部判断具体分量
保存粒度	以完整向量上下文为单位	以 <code>XSAVE</code> 状态分量为单位
观测方式	可在调度与保存路径上直接插桩	主要依据保存后的 <code>XSTATE_BV</code> 间接推断
主要边界	单一 <code>Dirty</code> 位不区分寄存器分量	分量级收益依赖相应状态是否被及时清理

因此，x86-64 的分量级追踪优势并非对所有扩展状态无条件成立。当某一分量缺少配套的显式清理路径时，`XSTATE_BV` 更接近“曾经使用”的历史标记，其判定效果可能退化为与 RISC-V 单一 `Dirty` 位相似的全有全无语义。两种架构在向量状态切换上的实际差异，应结合处理器性能、负载执行时间、编译器生成代码和具体状态分量共同分析，不宜仅根据单次试验的绝对保存次数得出结论。

附录 B 新内核与根文件系统适配过程

本章记录 Milk-V Jupiter（Spacemit K1，RISC-V）平台上的根文件系统与内核适配过程。适配目标是在保留 irradium 已完成板级支持的 OpenSBI、U-Boot、设备树和驱动基础上，将用户态替换为 openEuler，并编译 6.18.38 内核以启用 sched_ext 调度框架。整个过程不修改固件启动链，工作范围集中在根文件系统、内核配置、模块与固件部署以及 boot 分区指向关系，验证过程也围绕这些边界展开。

B.1 根文件系统适配

根文件系统适配采用“保留硬件支持、替换用户态”的方式。irradium 镜像继续提供与 K1 平台匹配的启动链、内核和板级驱动，openEuler rootfs 则提供新的用户空间。由于内核模块和固件与运行内核版本紧密关联，替换 rootfs 时必须同步保留与当前内核匹配的 /lib/modules 和 /lib/firmware，否则系统即使能够进入用户态，也可能因模块或固件缺失而无法驱动板载设备。

B.1.1 适配边界与输入材料

适配底包文件名如下。

irradium-3.8-riscv64-core-milk_v_jupiter-6.18.33-build-20260524.img.zst

该镜像首先写入 SD 卡，用于提供 6.18.33 内核及现有 K1 启动环境。用户态文件来自 openEuler-22.03-V2-riscv64/openeuler-rootfs.tar.gz。部署过程中保留原 boot 分区和旧 rootfs 分区，另行建立新分区承载 openEuler，以便在出现启动异常时保留原系统作为回退依据。

运行内核通过 `uname -r` 确定模块查找目录，因此新 rootfs 中的 /lib/modules/<ver> 必须与实际启动的内核版本一致。固件目录虽然不以版本号区分，但其内容同样需要满足板级驱动要求。基于这一约束，根文件系统适配不是单纯解压用户态文件，而是由分区准备、用户态写入、模块与固件迁移以及启动配置更新共同组成。

B.1.2 分区与根文件系统部署

分区规划。将 SD 卡接入虚拟机后，首先通过块设备信息确认目标设备，避免将后续操作作用于错误磁盘。适配过程保留原 boot 分区 /dev/sdb1 和旧 rootfs 分区 /dev/sdb2，在

缩小旧分区后新建 `/dev/sdb3`，并将其格式化为 `ext4`，作为 `openEuler` 的根文件系统分区。设备名应以实际识别结果为准，执行分区操作前需再次核对容量、分区表和挂载状态。

用户态与内核配套文件写入。新分区挂载到 `/mnt/irootfs` 后，先解压 `openEuler rootfs`，再从旧系统迁移内核模块与固件。解压命令使用 `-p` 保留文件权限，模块和固件使用归档模式复制，以维持符号链接、权限及目录结构。

```
mount /dev/sdb3 /mnt/irootfs
tar -xpf openeuler-rootfs.tar.gz -C /mnt/irootfs
rsync -av /mnt/oldrootfs/lib/modules/ /mnt/irootfs/lib/modules/
rsync -av /mnt/oldrootfs/lib/firmware/ /mnt/irootfs/lib/firmware/
```

根分区标识与卸载。完成文件写入后，通过 `blkid /dev/sdb3` 获取新分区 UUID，并将其写入新系统的 `/etc/fstab`。使用 UUID 而不是易受设备枚举顺序影响的设备名，可以使根分区标识在读卡器和板卡环境之间保持稳定。

```
UUID=<sdb3的UUID> / ext4 defaults,noatime 0 1
```

配置完成后，应同步文件系统并卸载所有已挂载分区，再将 SD 卡插回板卡启动。此时启动链与运行内核保持不变，仅根文件系统切换为 `openEuler` 用户态。

B.1.3 验证方法

根文件系统适配需要同时验证静态文件布局、模块版本关系和实际启动结果。表 B.1 按照“分区结构—文件完整性—内核配套文件—系统启动”的顺序给出检查项，使故障能够在进入板卡启动前尽量被定位。

表 B.1 根文件系统适配验证项

验证点	命令或依据	预期结果
分区结构	<code>lsblk</code>	三个分区的容量、文件系统类型与挂载关系正确
rootfs 完整性	<code>ls /mnt/irootfs</code>	存在 <code>bin</code> 、 <code>etc</code> 、 <code>lib</code> 、 <code>usr</code> 、 <code>var</code> 等标准目录
模块匹配	<code>ls /mnt/irootfs/lib/modules/</code>	存在与运行内核对应的 6.18.33 目录
fstab 正确性	<code>blkid /dev/sdb3</code> 与 <code>fstab</code> 对照	根分区 UUID 与 <code>fstab</code> 记录完全一致
启动结果	串口启动日志	出现 <code>openEuler 22.03 LTS</code> 与 <code>Kernel 6.18.33</code> 登录提示
最终确认	<code>uname -a</code>	<code>openEuler</code> 用户态运行于 <code>irradium 6.18.33 RISC-V</code> 内核

上述检查形成由离线到在线的验证链。分区与文件布局用于确认写入过程完整，模块目录用于确认内核与用户态之间的版本关系，串口日志和 `uname -a` 则用于确认板卡确实从目标根文件系统进入运行状态。只有这些条件同时成立，才能认为根文件系统替换完成。

B.2 新内核适配与 sched_ext 启用

在 openEuler 根文件系统能够稳定启动后，进一步基于 K1 内核源码编译 6.18.38 内核，并增量启用 sched_ext 所需配置。该阶段继续沿用已有板级配置，避免在验证新调度框架的同时引入无关的驱动差异；部署范围包括内核镜像、模块、固件以及 boot 分区中的当前内核链接。

B.2.1 配置目标与编译输入

irradiation 自带的 6.18.33 内核未启用 CONFIG_SCHED_CLASS_EXT，无法注册 sched_ext 调度器。新内核以 config-6.18.33 作为基础配置，在保留 K1 既有设备支持的前提下，开启 BPF、BPF JIT、BTF 和 sched_ext 相关选项。该方式将配置变更限制在目标特性所需范围内，便于在启动异常时区分板级配置问题与新增调度功能问题。

内核源码包：kernel-source-k1#6.18.38-1.pkg.tar.gz

配套固件包：kernel-firmware-k1#6.18.38-1.pkg.tar.gz

构建沿用 config-6.18.33 基础配置，并采用前缀为 riscv64-linux-gnu- 的 RISC-V 交叉编译工具链；内核镜像、模块和设备树必须来自同一构建版本。

B.2.2 内核配置与编译

配置继承与特性启用。将 config-6.18.33 复制为内核源码树下的 .config，再通过 make menuconfig 核对 sched_ext 及其依赖项。最终启用的关键配置如下。

```
CONFIG_BPF=y
CONFIG_BPF_SYSCALL=y
CONFIG_BPF_JIT=y
CONFIG_BPF_JIT_ALWAYS_ON=y
CONFIG_BPF_JIT_DEFAULT_ON=y
CONFIG_SCHED_CLASS_EXT=y
CONFIG_DEBUG_INFO_BTF=y
CONFIG_PAHOLE_HAS_SPLIT_BTF=y
```

CONFIG_DEBUG_INFO_BTF 依赖 pahole 工具；CONFIG_PAHOLE_HAS_BTF_TAG 还需要 clang 支持，本次构建未启用该选项。配置核对的重点不仅是确认 CONFIG_SCHED_CLASS_EXT=y，还要保证 BPF 系统调用、JIT 和 BTF 信息能够共同满足 sched_ext 程序的加载条件。

内核、模块与设备树编译。编译阶段显式指定 RISC-V 架构和交叉编译器，并生成内核镜像、模块与设备树。构建完成后，应首先检查目标版本号和关键产物，再进入 SD 卡安装阶段。

```
make ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu- -j8 modules dtbs Image
make ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu- -j8
```

B.2.3 产物部署与启动项切换

内核与模块安装。将 boot 分区和新 rootfs 分别挂载到 /mnt/iboot 与 /mnt/irootfs，随后安装模块、内核镜像和配套固件。为降低直接写入 SD 卡的风险，可先将安装路径指向临时目录核对产物，确认文件版本和目录结构无误后再执行正式安装；具体位置由下述命令中的 INSTALL_MOD_PATH 与 INSTALL_PATH 指定。

```
mount /dev/sdb1 /mnt/iboot
mount /dev/sdb3 /mnt/irootfs
make ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu- \
    modules_install INSTALL_MOD_PATH=/mnt/irootfs
make ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu- \
    install INSTALL_PATH=/mnt/iboot
tar -xf 'kernel-firmware-k1#6.18.38-1.pkg.tar.gz' -C /mnt/irootfs
```

boot 分区链接更新。boot 分区通过符号链接指向当前使用的内核镜像、映射文件、配置文件和设备树目录。安装 6.18.38 产物后，需要统一更新这些链接，并将原 initrd.img 改名停用，使系统直接从分区中的 rootfs 启动。

```
cd /mnt/iboot
ln -sf vmlinuz-6.18.38 Image
ln -sf vmlinuz-6.18.38 vmlinuz
ln -sf System.map-6.18.38 System.map
ln -sf config-6.18.38 config
rm dtb && ln -s dtbs/6.18.38 dtb
mv initrd.img initrd.img.disable
```

若 `vmlinuz-6.18.38`、`dtbs/6.18.38/` 或其他目标文件缺失，应返回构建产物检查环节补齐文件，不应保留指向不存在目标的符号链接。完成链接核对后，卸载分区并将 SD 卡插回板卡引导。

B.2.4 验证方法

新内核适配的验收需要覆盖启动产物、模块目录、内核配置和 `sched_ext` 运行时接口。表 B.2 将静态文件检查与启动后的动态检查分开列出，便于判断故障发生在安装阶段、内核引导阶段还是功能注册阶段。

表 B.2 新内核适配验证项

验证点	命令或依据	预期结果
boot 链接	<code>ls -l /mnt/iboot</code>	<code>Image</code> 、 <code>dtb</code> 指向 6.18.38 对应产物， <code>initrd.img</code> 已停用
内核版本	<code>uname -r</code> （启动后）	输出 6.18.38
模块匹配	<code>ls /lib/modules/\$(uname -r)</code>	版本目录存在且非空，模块能够由 <code>modprobe</code> 正常加载
配置生效	<code>grep CONFIG_SCHED_CLASS_EXT /boot/config-\$(uname -r)</code>	输出 <code>CONFIG_SCHED_CLASS_EXT=y</code>
<code>sched_ext</code> 运行时	<code>cat /sys/kernel/sched_ext/state</code>	接口文件存在，并输出 <code>enabled</code> 或 <code>disabled</code>

验证结果必须同时满足三个层次：6.18.38 内核能够完成引导，目标配置与模块版本关系正确，`sched_ext` 运行时接口已经注册。任一层次缺失，都只能说明部分安装步骤完成，不能据此认定自定义调度框架已经具备可用条件。通过这一分层验收，可以将内核适配结果与正文中的 `sched_ext` 原型功能验证建立清晰对应关系。

B.2.5 复现边界与维护要求

内核升级后的可复现性依赖成套产物，而不是单独的内核镜像。后续更新版本时，应同步记录源码包、配置差异、工具链、设备树、模块与固件，并保证 `boot` 链接和 `rootfs` 模块版本一致。仅替换 `Image` 可能使系统完成早期引导，却在设备初始化或模块加载阶段出现故障。

本附录的 6.18.38 环境仅用于验证 `sched_ext` 在 K1 平台具备运行条件；正文六种通用调度方法用于受控基线下的正式性能比较，两类证据不作横向混用。若需比较，应统一内核、根文件系统、模型、CPU 配置、观测脚本、重复次数与统计口径后重新采集。